



Unity를 이용한 2D 게임프로그래밍

Lecture 11

2D 게임프로그래밍 정리

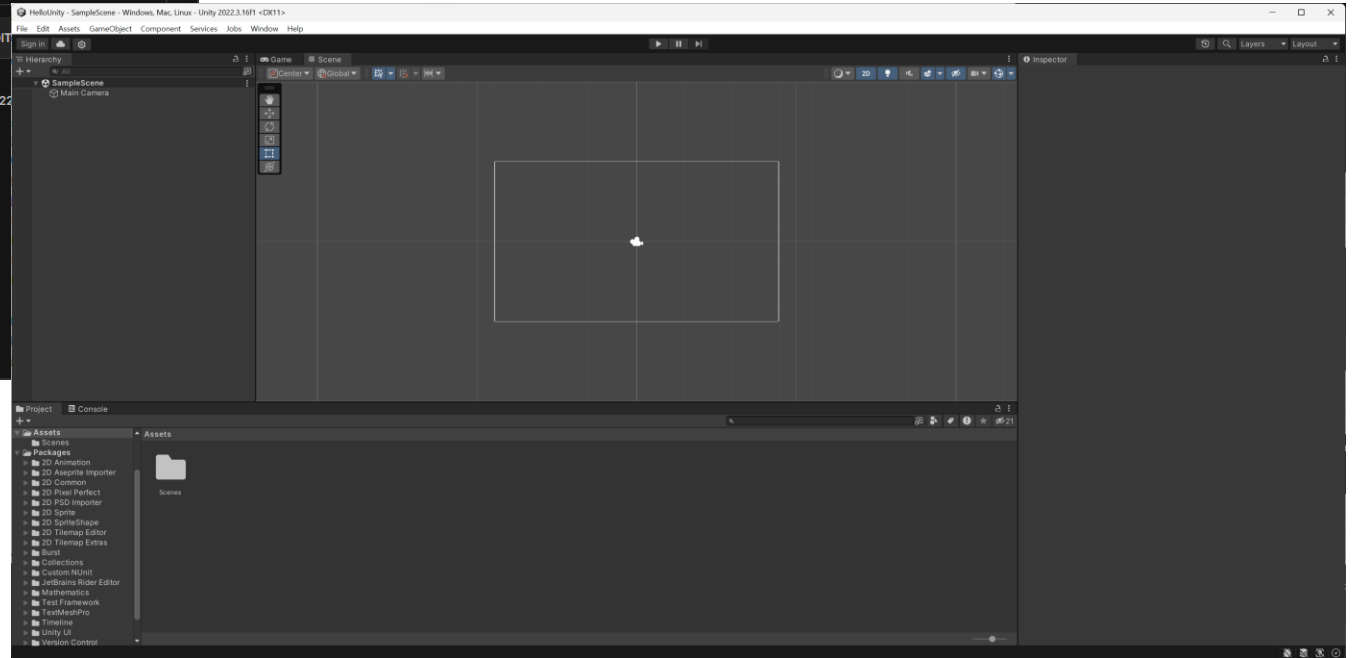
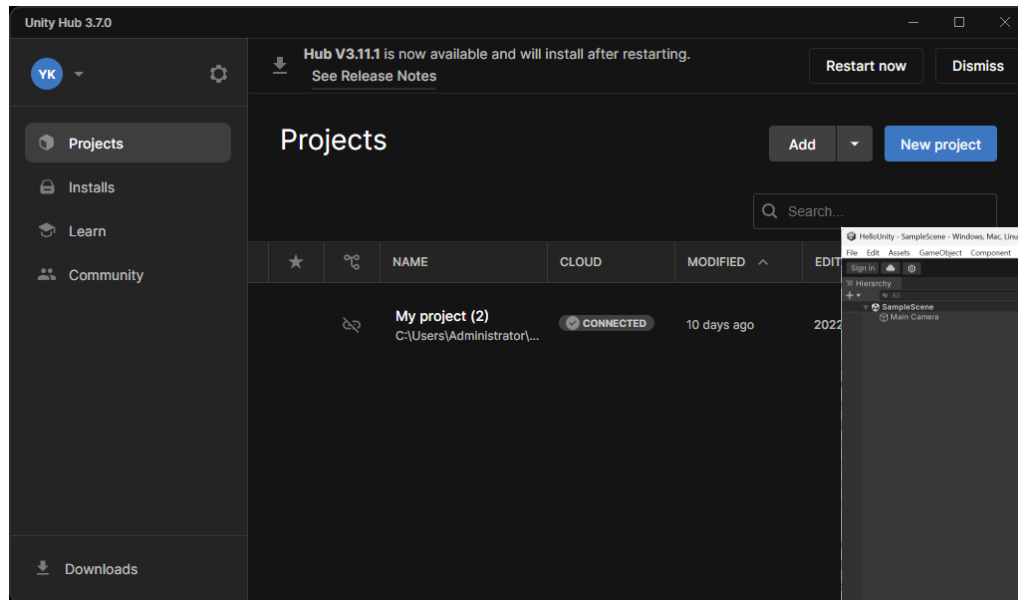
강영민

동명대학교 게임공학과

Unity Hub – Unity Engine

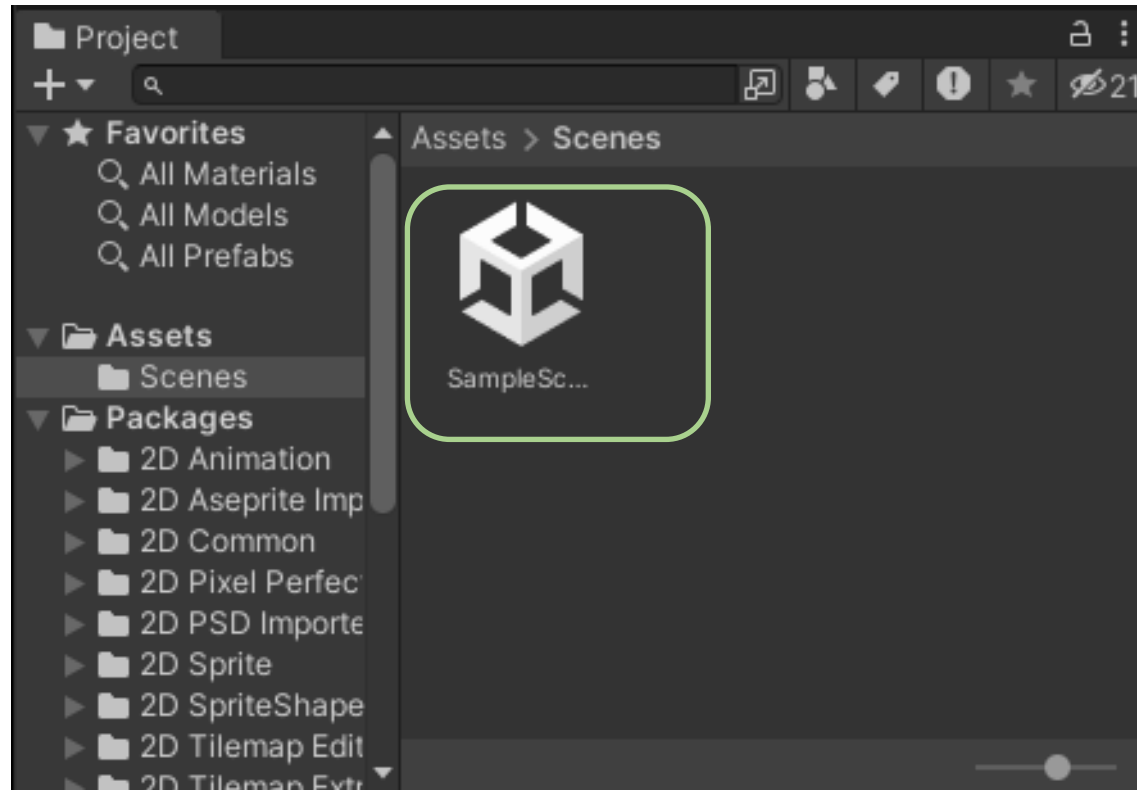
• 허브

엔진



게임 개발 기본 바탕 - SampleScene

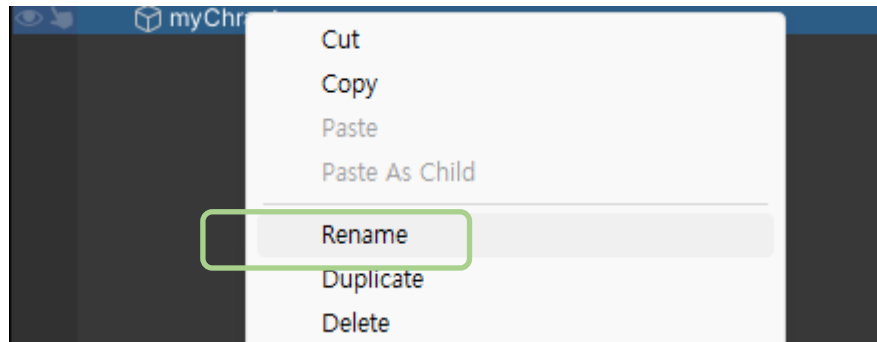
- 유니티 프로젝트를 생성하면 SampleScene이 생성됨



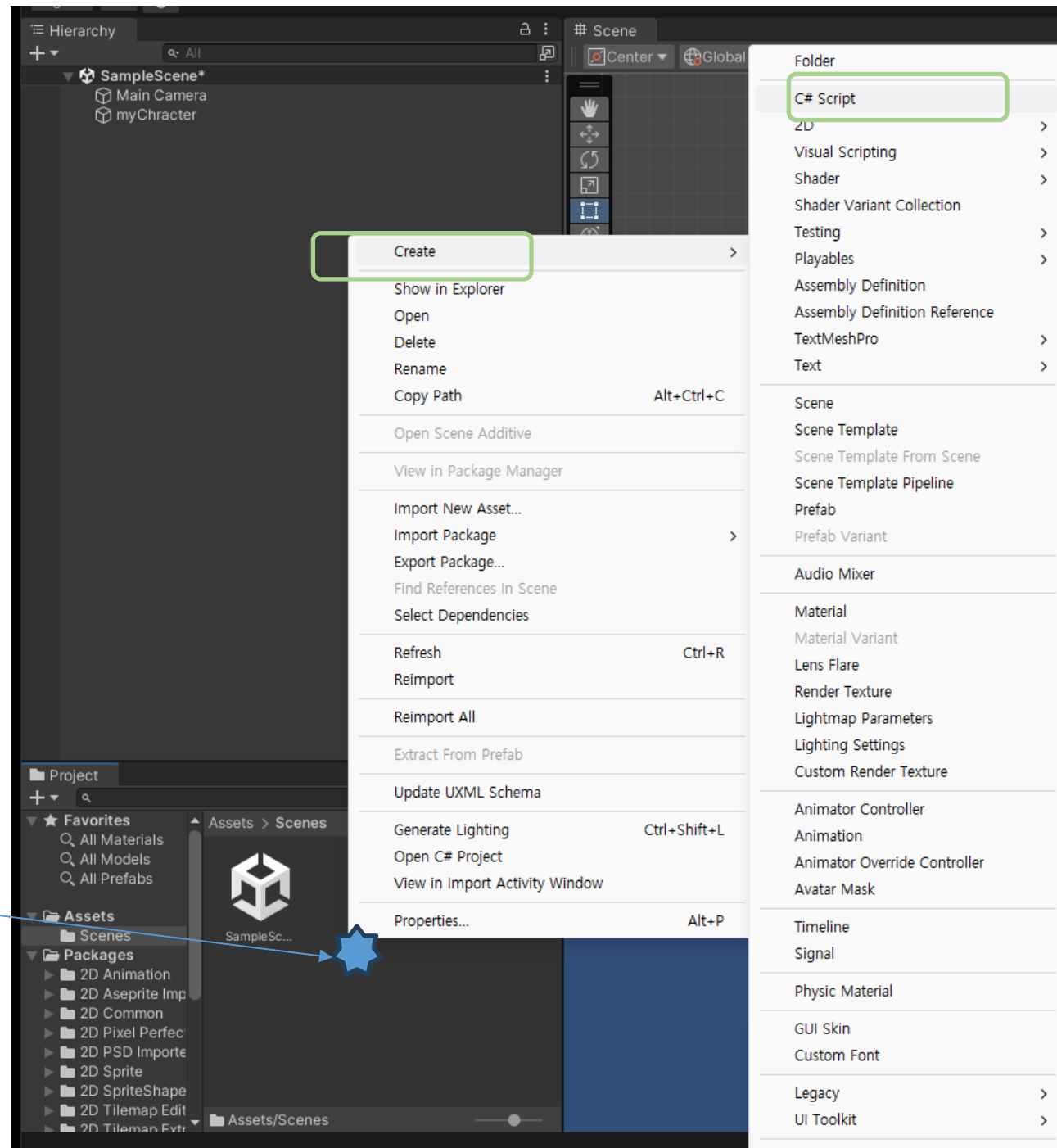
스프라이트 움직이기 위한 “method” 사용

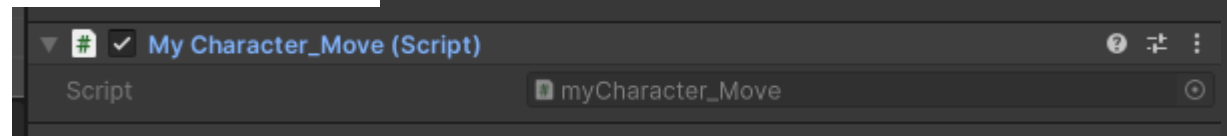
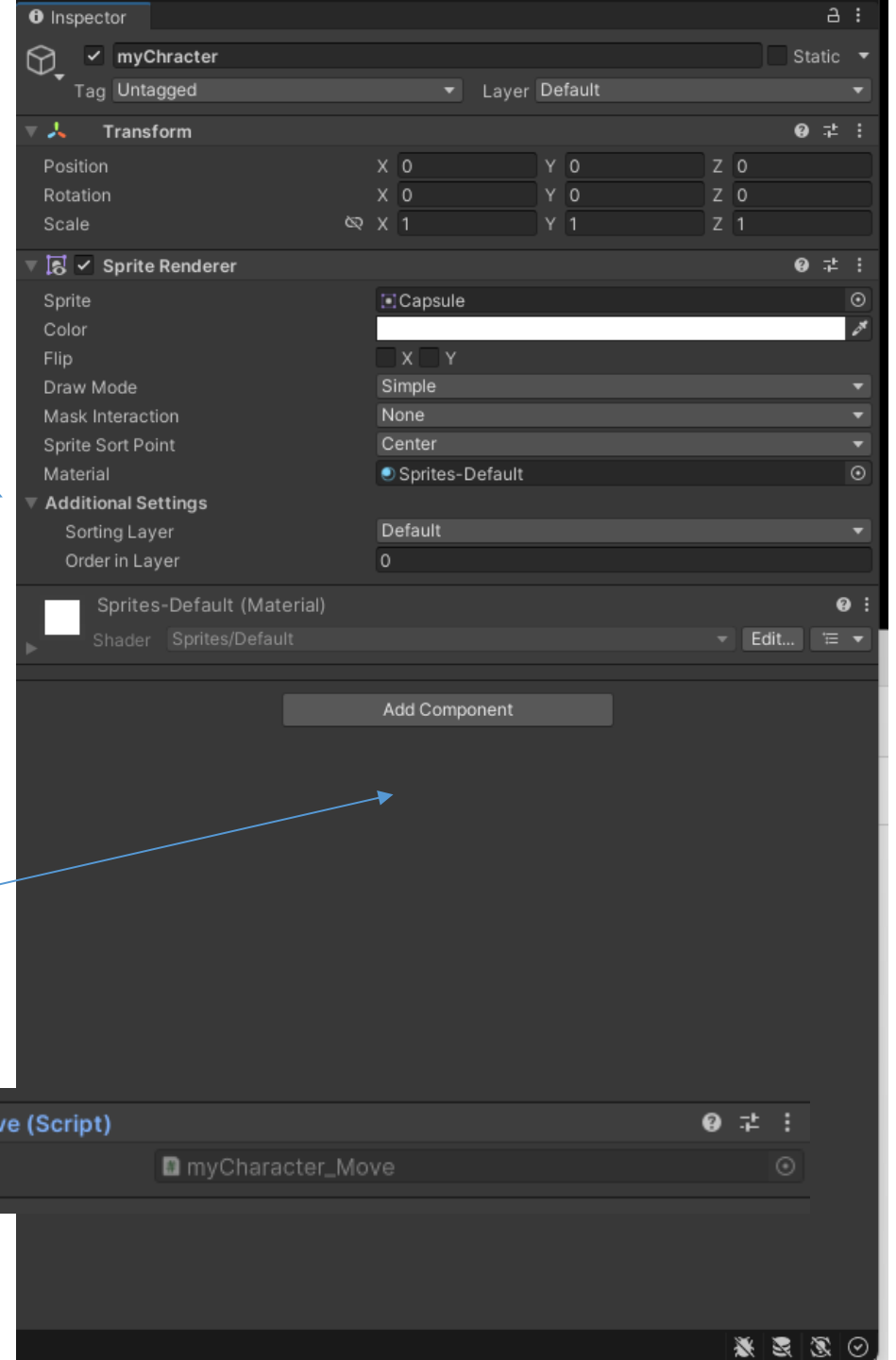
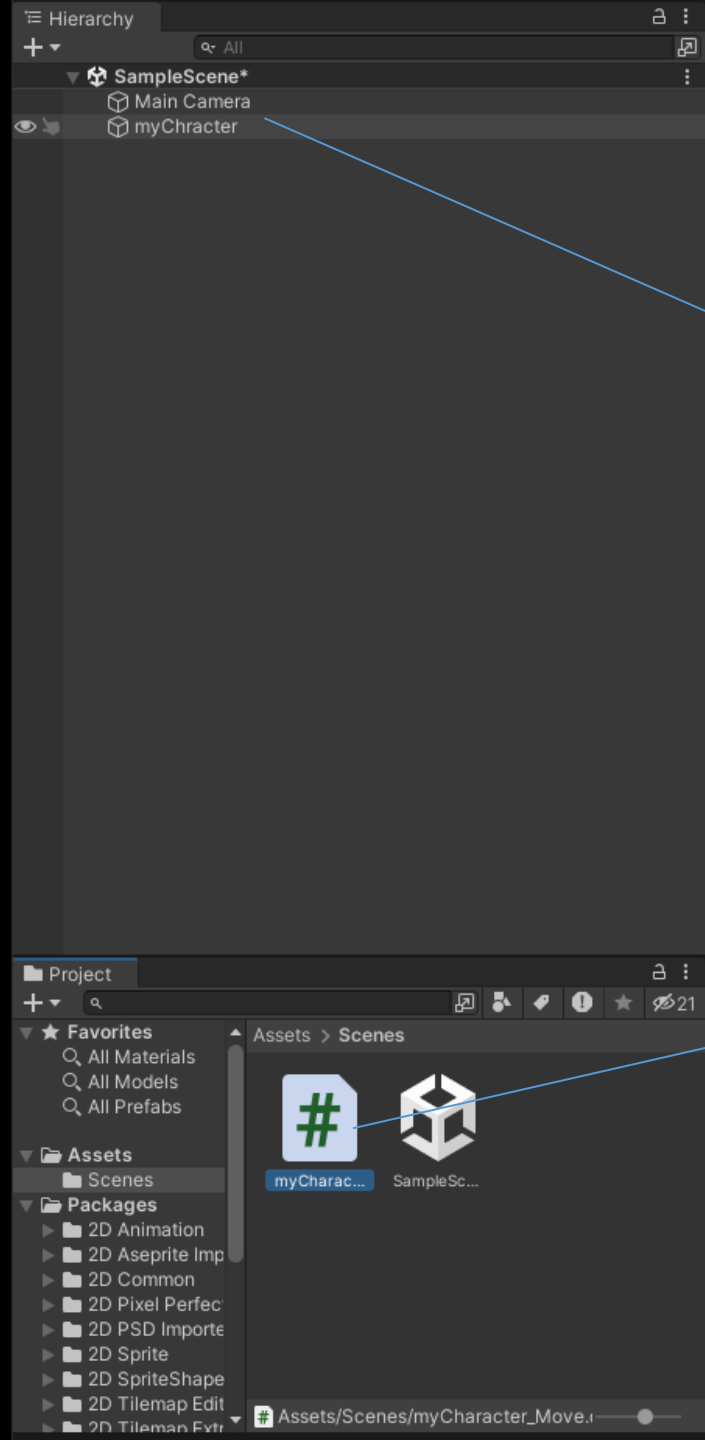
- 유니티의 메소드
 - 우리의 게임이 무엇인가를 할 수 있도록 하는 코드의 덩어리를 실행시키는 도구
 - 함수와 비슷한 개념 – 메소드는 객체에 딸린 함수
- 사용가능한 메소드
 - 유니티에서 제공하는 메소드
 - 우리가 직접 만든 메소드
- 메소드를 만들고 실행하는 것
 - 만들기: 무엇인가를 할 수 있는 코드를 메소드로 작성해 두는 것
 - 실행: 만들어 놓은 메소드를 호출하여 해당 동작이 이루어지게 하는 것

스크립트를 연결하기

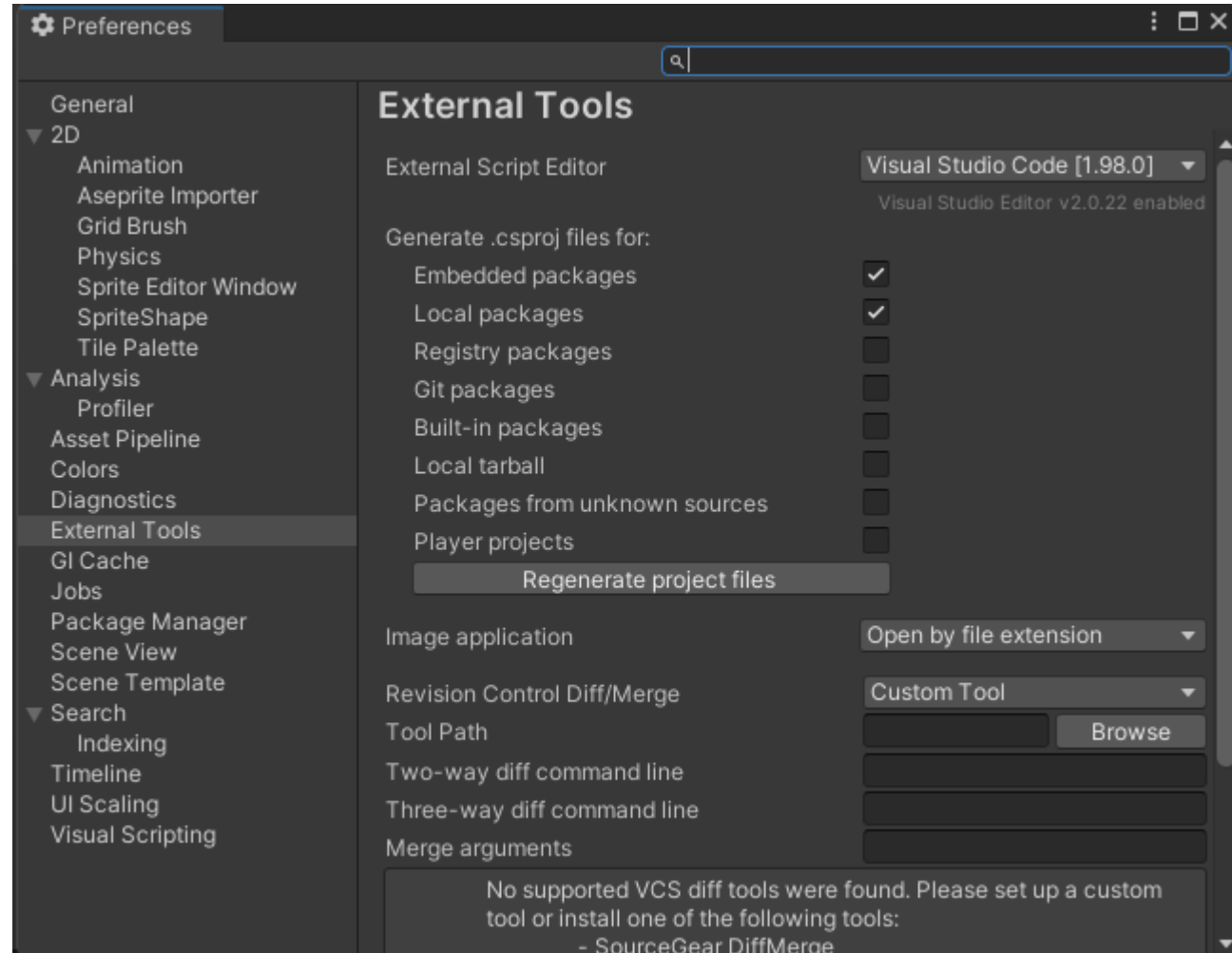


Right click!

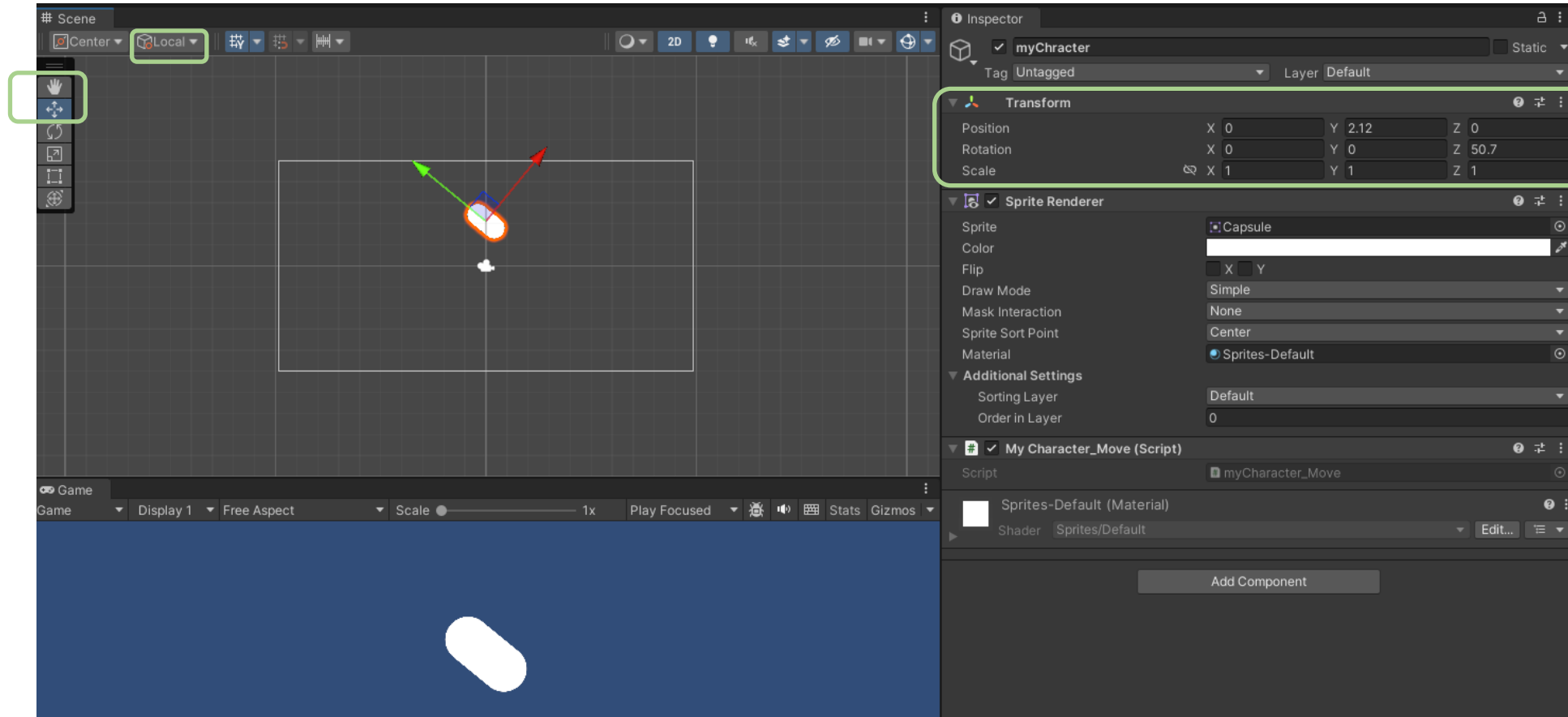




Unity Preference



인스펙터(Inspector)로 값의 변화 확인



Input System 사용하기

- 입력 시스템: 사용자의 물리적 동작을 게임에 필요한 정보로 변경
 - 물리적 동작: 버튼 누르기, 키 누르기, 조이스틱 조작 등
- 현재의 유니티는 두 종류의 입력 시스템을 가짐
 - Old system
 - New system
- 이 프로젝트에서는 old system을 사용해 보자

Input System 사용시 Axis의 정확한 이름 필요

The screenshot shows the Unity Input Manager interface. It is divided into two sections: Horizontal and Vertical. Each section has a list of properties on the left and their corresponding values on the right. The 'Name' property for both axes is highlighted with a green box.

Property	Horizontal Value	Vertical Value
Name	Horizontal	Vertical
Descriptive Name		
Descriptive Negative Name		
Negative Button	left	down
Positive Button	right	up
Alt Negative Button	a	s
Alt Positive Button	d	w
Gravity	3	3
Dead	0.001	
Sensitivity	3	
Snap	☒	
Invert	☐	
Type	Key or Mouse Button	
Axis	X axis	
Joy Num	Get Motion from all Joysticks	

Axis의 신호를 스크립트에서 얻을 수 있음

`Input.GetAxis(name)`

캐릭터 제어

게임이란 무엇이고?
입력시스템의 역할은?

```
C# myCharacter_move.cs X
Users > ymkang > CarGame > Assets > Scripts > myCharacter_move.cs > myCharacter_move > moveSpeed
1  using UnityEngine;
2
3  0 references
4  public class myCharacter_move : MonoBehaviour
5
6  1 reference
   [SerializeField] float steerSpeed = 0.5f;
7  1 reference
   [SerializeField] float moveSpeed = 0.01f;
8  // Start is called once before the first execution of Update after the MonoBehaviour is created
9  0 references
   void Start()
10 {
11 }
12
13 // Update is called once per frame
14 0 references
   void Update()
15 {
16
17     float steerAmount = -Input.GetAxis("Horizontal");
18     float moveAmount = Input.GetAxis("Vertical")*moveSpeed;
19     transform.Rotate(0, 0, steerSpeed * steerAmount);
20     transform.Translate(0, moveAmount, 0);
21 }
22
23
```

싱글톤 접근성을 지원하는 게터

```
public class GameManager : MonoBehaviour
{
    private static GameManager instance;
    //싱글톤 접근용 프로퍼티
    public static GameManager Instance
    {
        get{
            if(instance == null)
            {
                return null;
            }
            return instance;
        }
    }
}
```

컴퓨터 성능에 따라 다른 동작

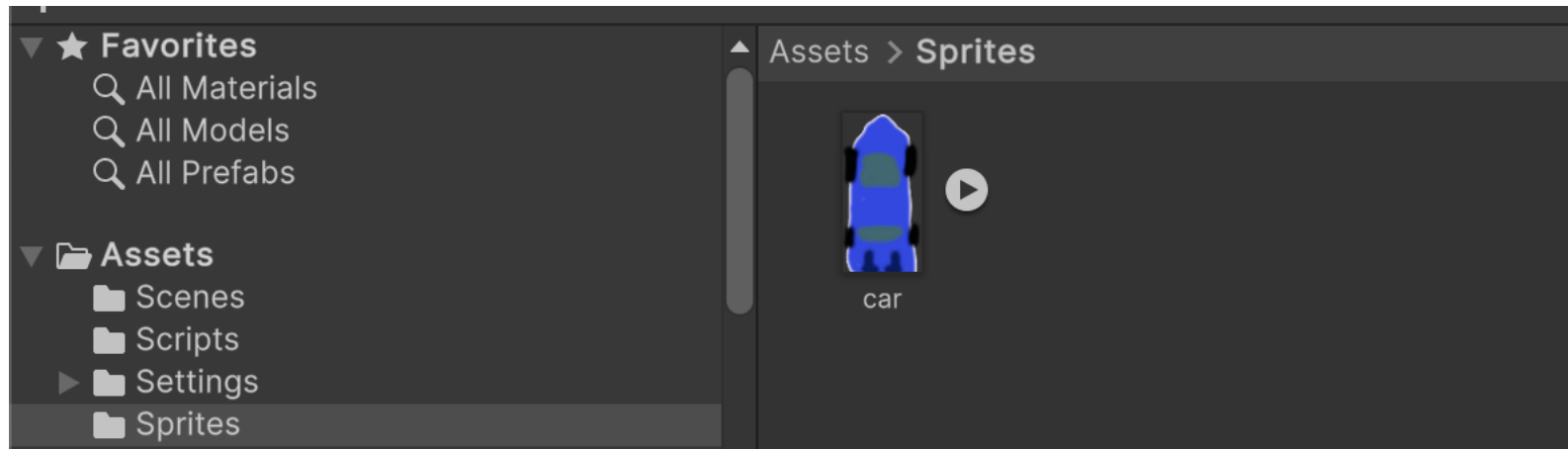
- 고성능 컴퓨터
 - Update 메소드가 더 자주 불리게 된다
 - 이는 변화가 더 많이 일어나서 캐릭터가 더 빨리 움직이게 됨
 - 이를 해결하는 방법
 - 고성능 컴퓨터에서는 Update 안에서 조금만 움직이고
 - 저성능 컴퓨터에서는 Update 안에서 더 많이 움직인다
 - 어떻게 이를 구현할 것인가?
 - 이전 Update에서 이번 Update가 불리는 데에 걸린 시간
 - Time.deltaTime
 - 고성능 컴퓨터: 작은 deltaTime
 - 저성능 컴퓨터: 큰 deltaTime
-  deltaTime을 변화의 규모에 곱하면!!

Trigger – Collider의 차이

- Collider
 - 물리적인 충돌 반응이 나타남
 - Rigidbody인 경우
 - 물체가 강체의 특성에 따라 움직임
 - Rigidbody가 아닌 경우
 - Static한 물체로 제자리에 머물며 상대를 밀어냄
- Trigger
 - 물리적인 충돌 반응이 없음
 - 단순히 두 물체가 겹쳤음을 감지
 - 감지 이후의 동작은 OnTriggerEnter2D로 처리

Asset

Assets 폴더 아래 자유롭게

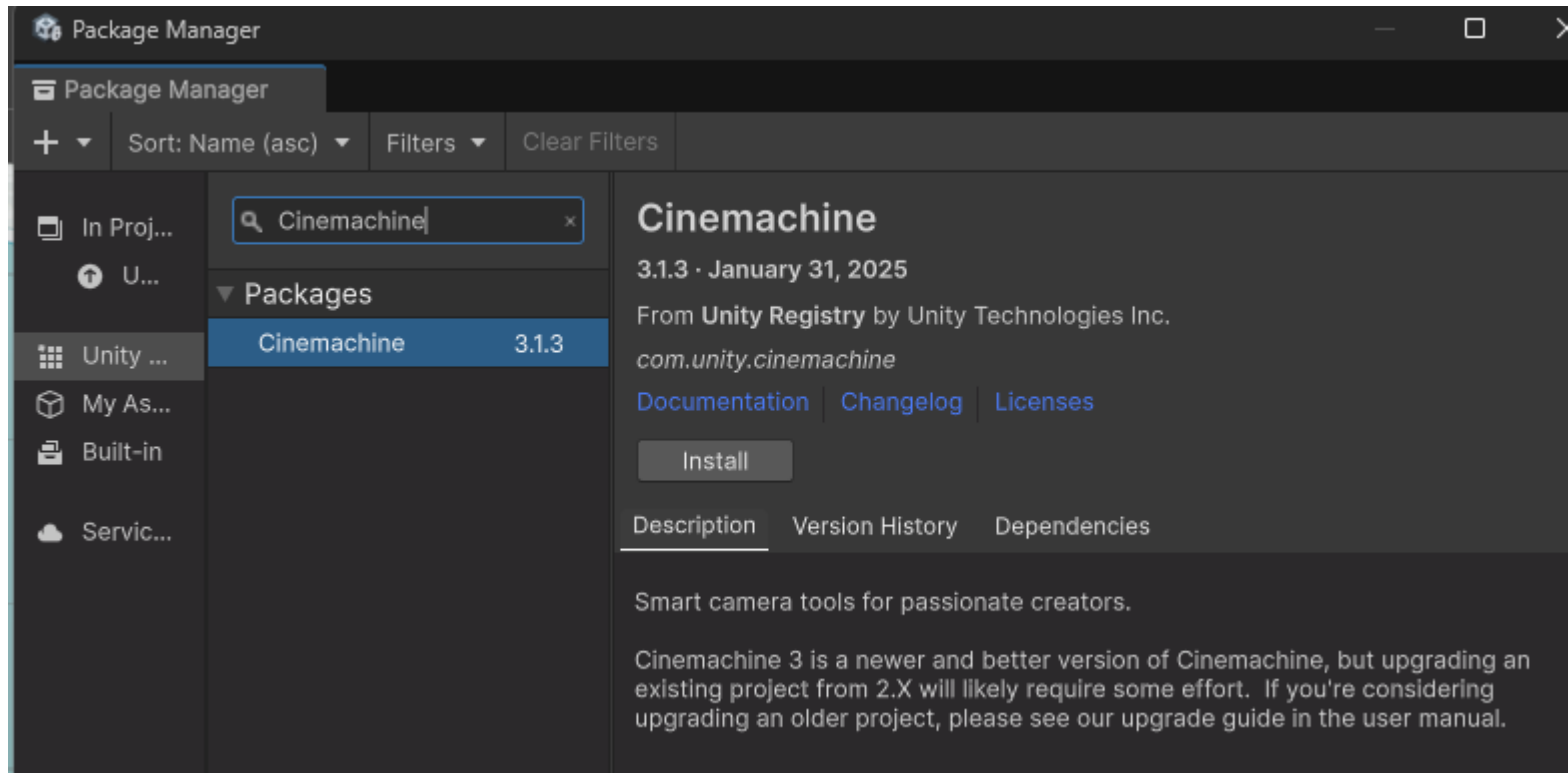


씨네머신

- 다수의 카메라를 장면에 도입할 수 있게 함
- 카메라에 적용되는 규칙을 간단히 정의할 수 있음
- 씨네머신이 다수의 가상 카메라를 제어
- 규칙에 따라 카메라 선택과 동작 제어
- 씨네머신: 패키지의 일종
- 사용하고 싶으면 프로젝트에 추가해야 함

씨네머신 패키지 사용

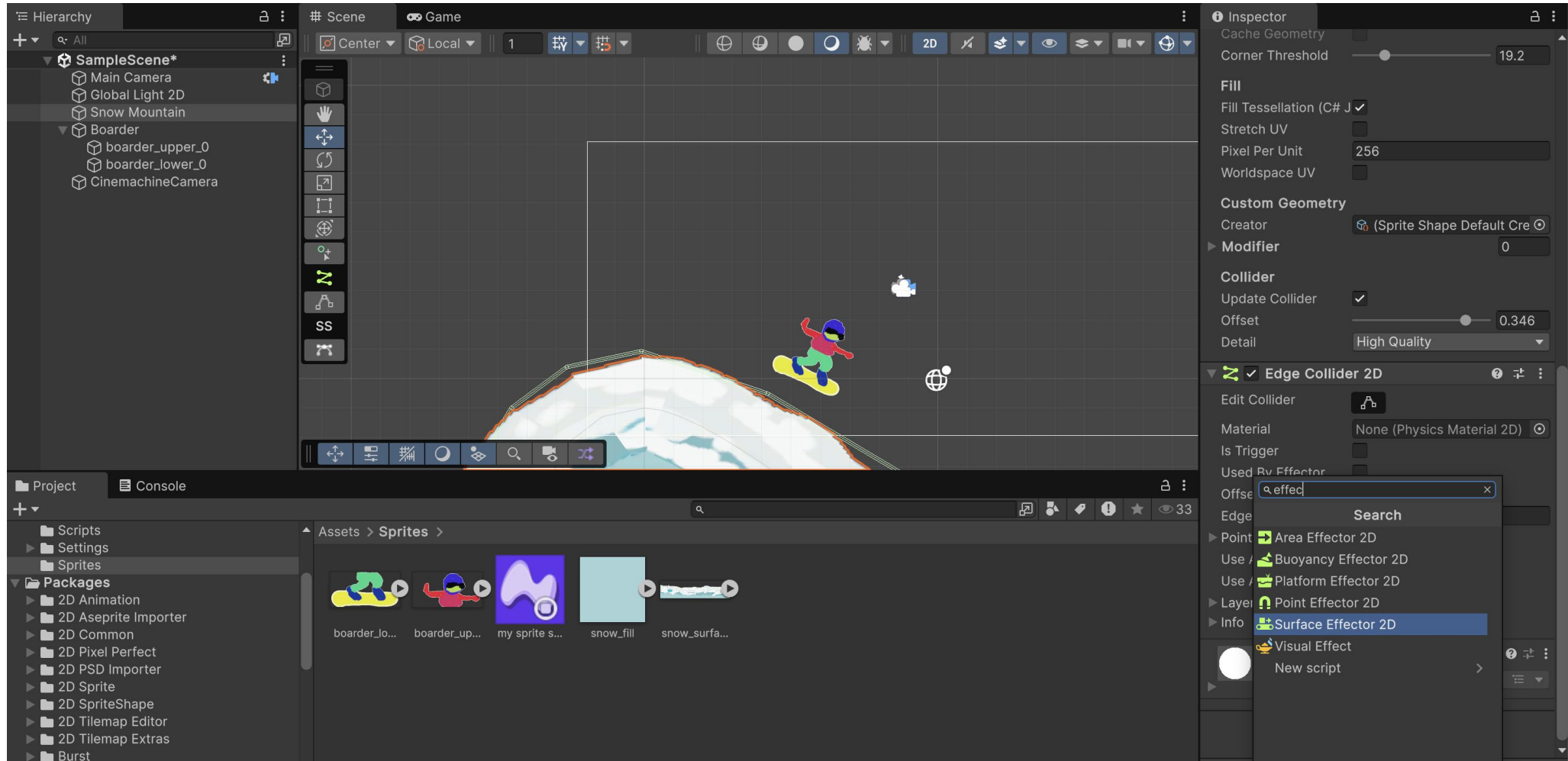
Windows → Package Manager → Unity Registry → Search cinemachine → Install!



Surface Effector

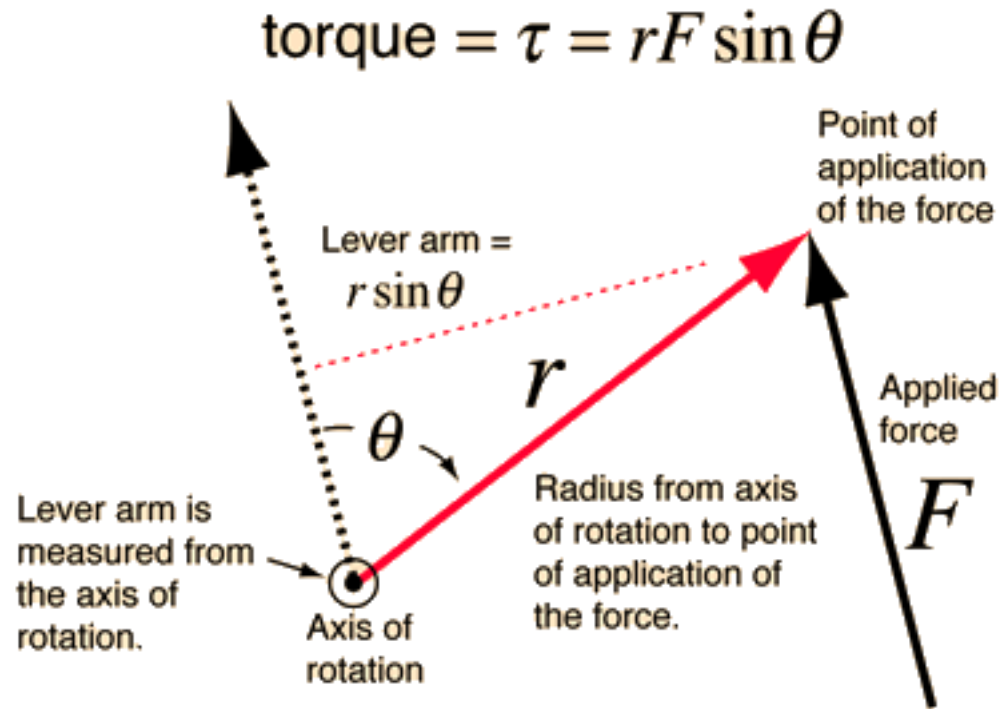
- Collider 표면에서 힘이 발생하게 함
 - 콜라이더 표면이 일정한 컨베이어 벨트처럼 되게 할 수 있음
- 이펙터의 종류
 - 서피스 이펙터
 - 부력 이펙터
 - 포인트 이펙터 등

설산에 이펙터 컴포넌트 추가



보더가 앞 뒤로 회전할 수 있게 하자

회전은 토크로 바꿀 수 있다



토크 표현방법:

회전력의 크기 x 회전 축

2D 게임에서는 언제나 z축 회전
(0, 0, t)

Board에 스크립트 연결

0 references

```
public class PlayerRotate : MonoBehaviour
```

2 references

```
[SerializeField] float
```

3 references

```
Rigidbody2D rb2d;
```

0 references

```
// Start is called once the object is instantiated and before the first frame update.
```

0 references

```
void Start()
```

```
{  
    rb2d = GetComponent<Rigidbody2D>();  
}
```

0 references

```
// Update is called once per frame
```

0 references

```
void Update()
```

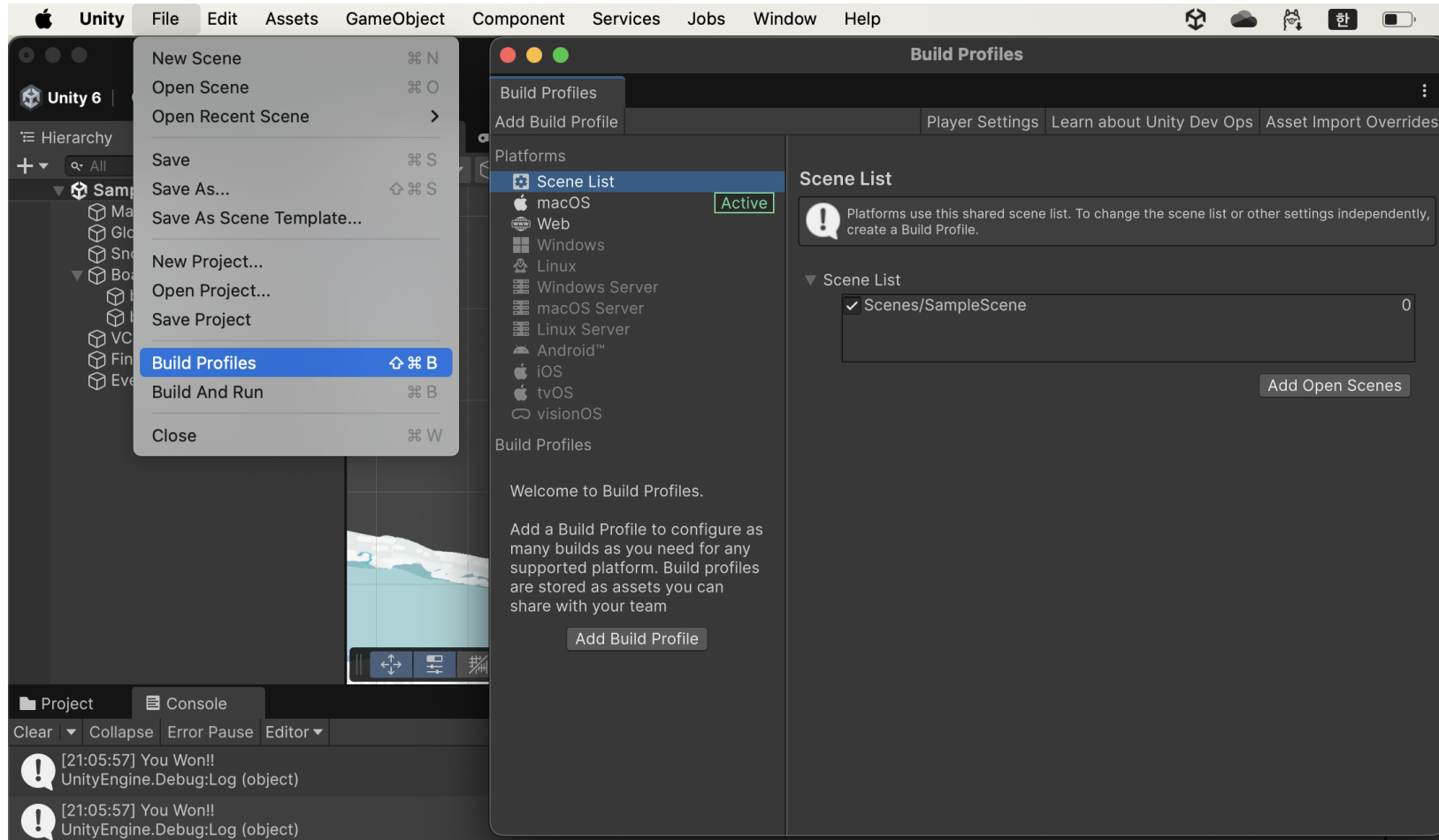
```
{  
    if (Input.GetKey(KeyCode.Space))  
    {  
        rb2d.AddTorque(torqueAmount*Time.deltaTime);  
    }  
    if (Input.GetKey(KeyCode.RightArrow))  
    {  
        rb2d.AddTorque(-torqueAmount*Time.deltaTime);  
    }  
}
```

Rigidbody 2D

Body Type	Dynamic
Material	None (Physics Material 2D)
Simulated	☒
Use Auto Mass	☐
Mass	1
Linear Damping	0.1
Angular Damping	1
Gravity Scale	1
Collision Detection	Continuous
Sleeping Mode	Start Awake
Interpolate	None

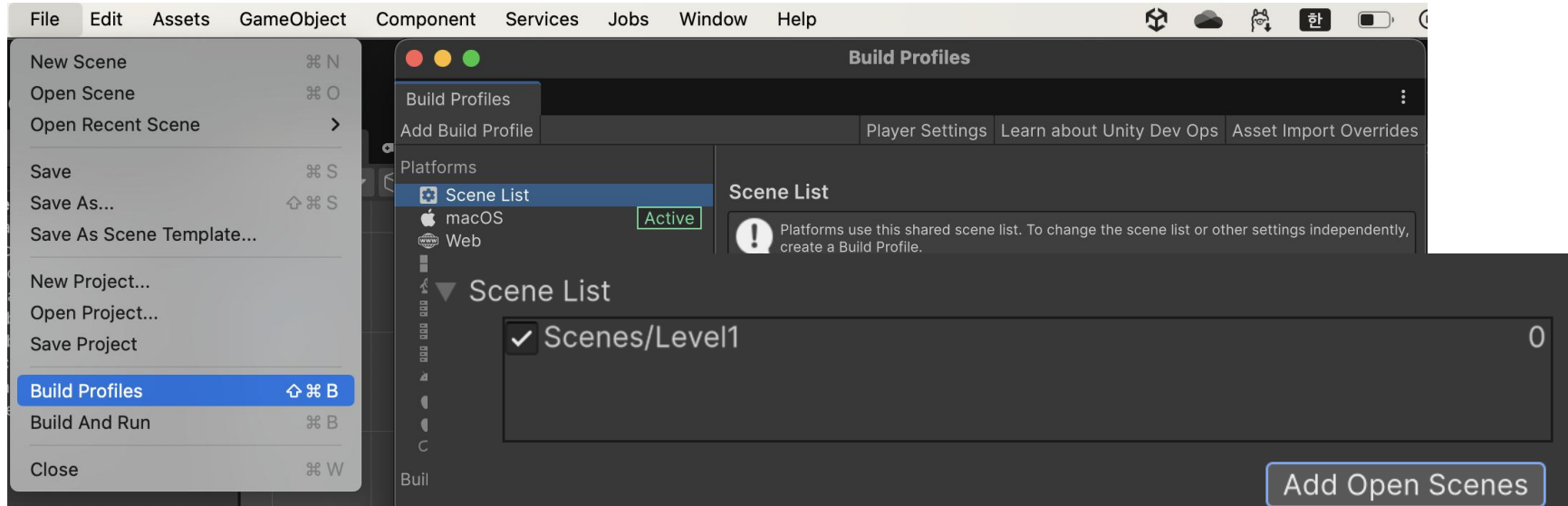


Scene 관리



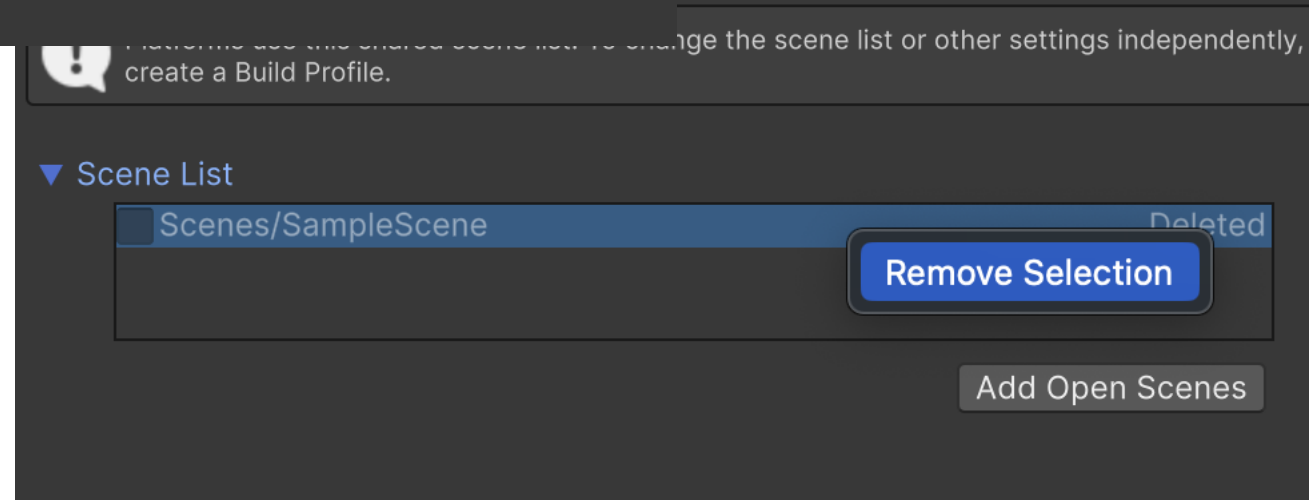
File → Build Profile → Scene List : SampleScene 하나가 있는 것을 확인

Build Profile 다시 확인



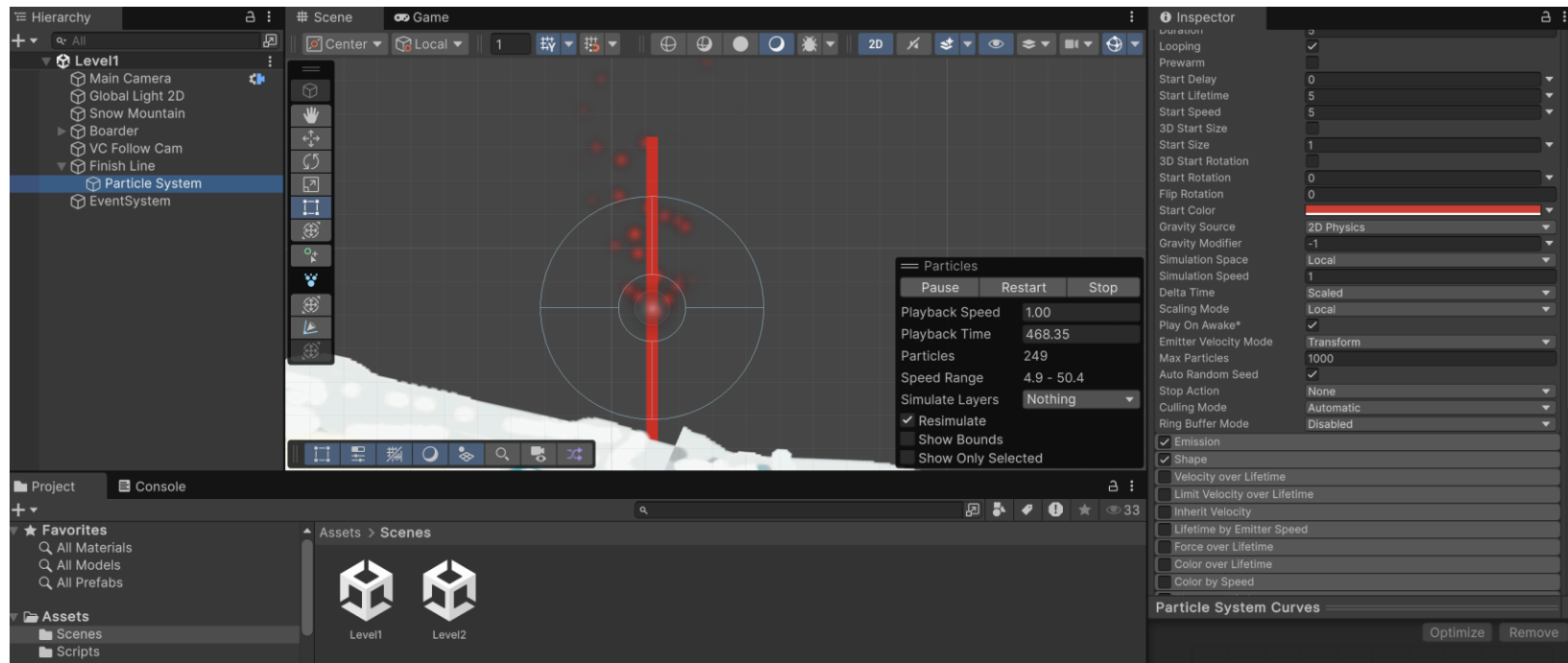
SampleScene0이 회색으로 표시되고 지워졌다는 의미입니다

Remove하자



입자 시스템 (Particle System)

Create → Effect → Particle System을 Finish Line 객체에 달아 보자

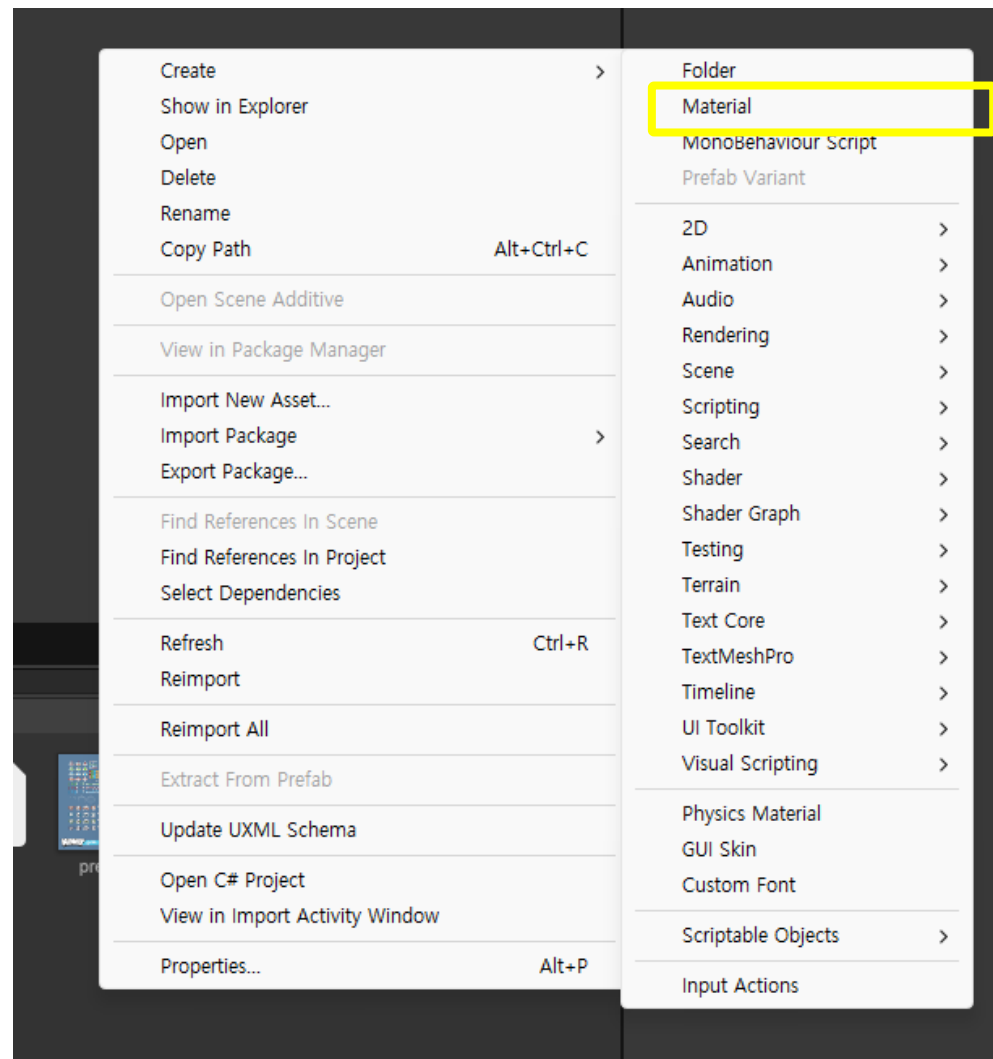
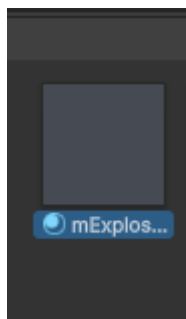


폭발 효과 – Particle System

- Particle System 생성
 - 입자 효과 구현
- Prefab으로 만들기
- 필요할 때마다 Instantiate

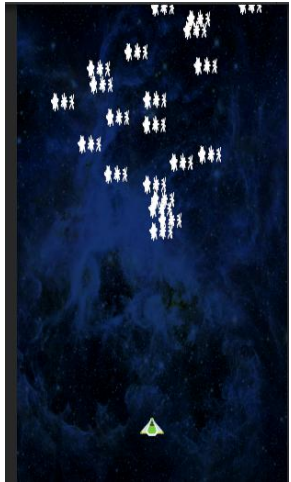
입자 시스템에 적용할 “재질-material” 만들기

- Asset 폴더에 Material 생성

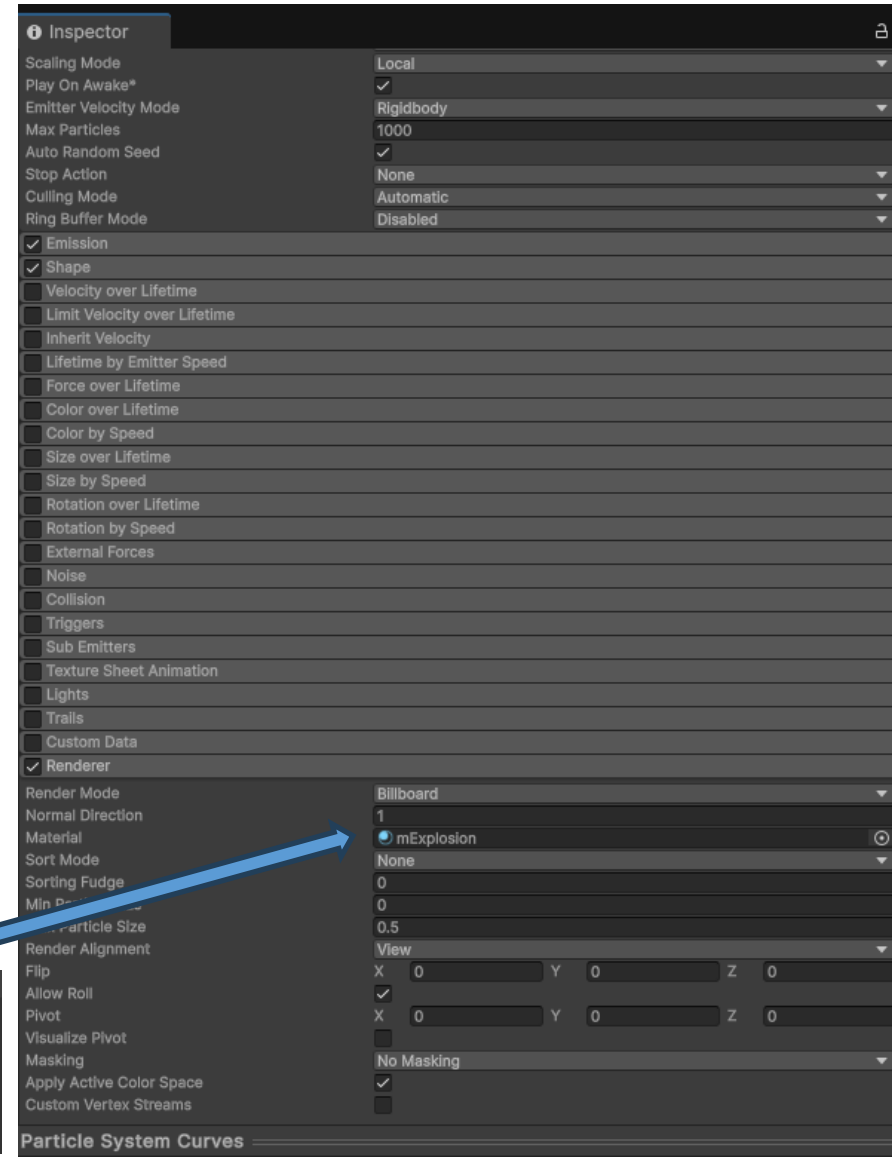
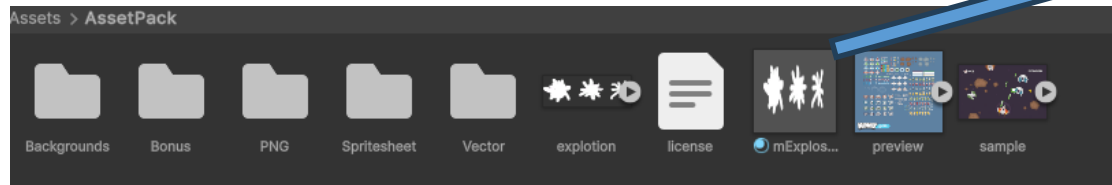


Particle System에 만든 재질을 적용

Particle System의 Inspector
→ Renderer → Material

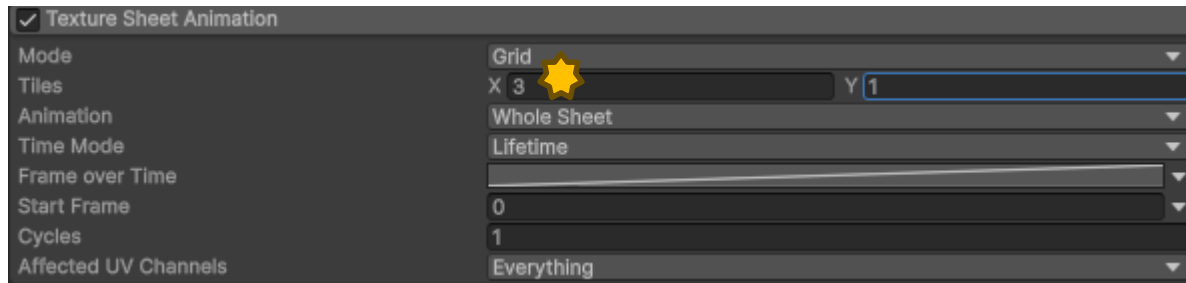


만든 재질을 가져다 설정

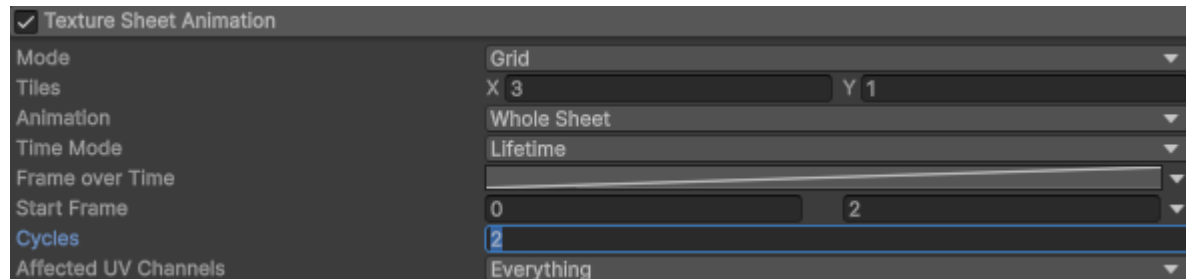


Particle System 설정 변경

Sprite Sheet에서 한 영역만 가져오기: 스프라이트 시트를 3개의 타일로 변경

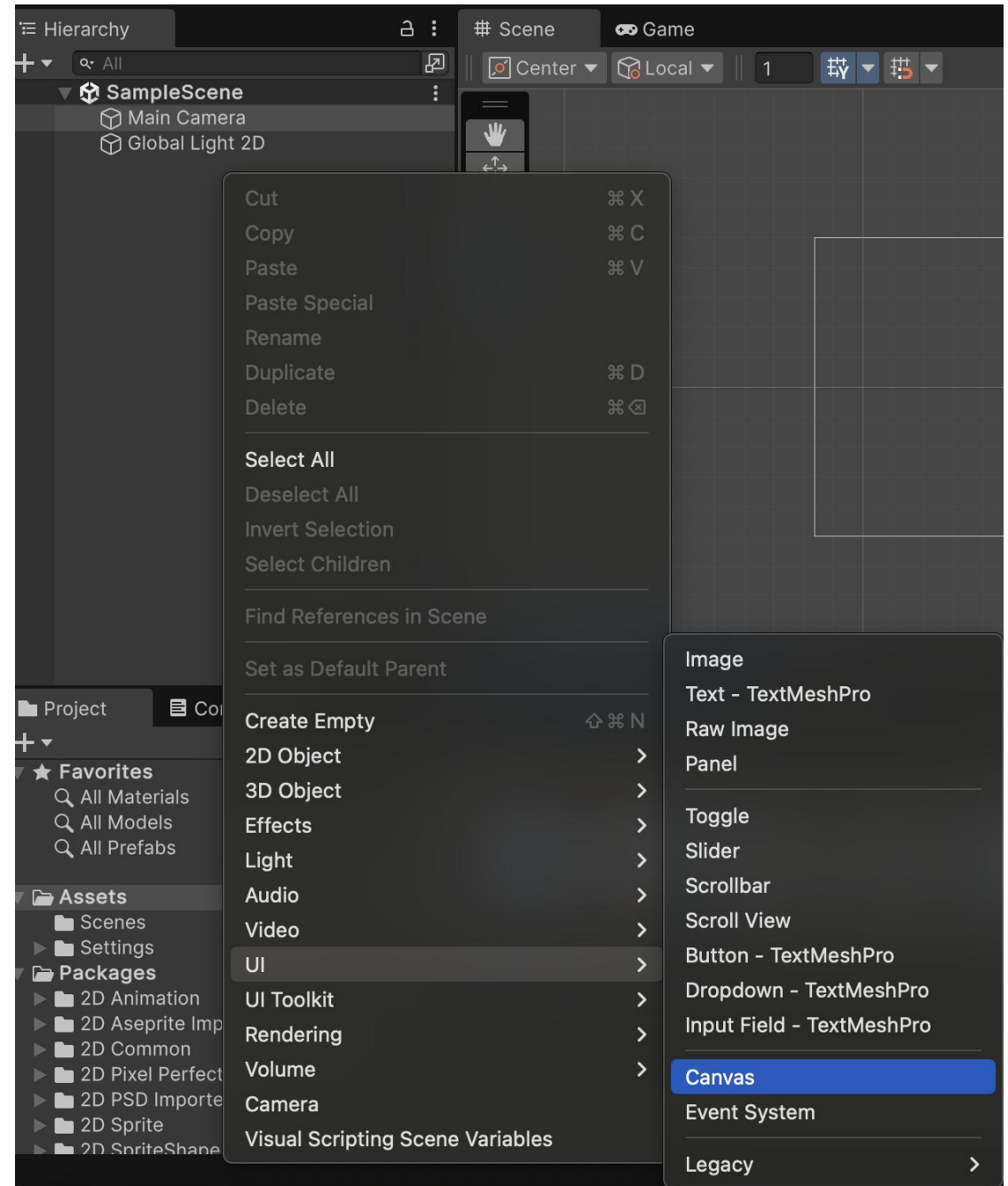


파티클이 존재하는 동안 두 번의 사이클로 스프라이트 애니메이션 실행 + 시작 프레임 랜덤화



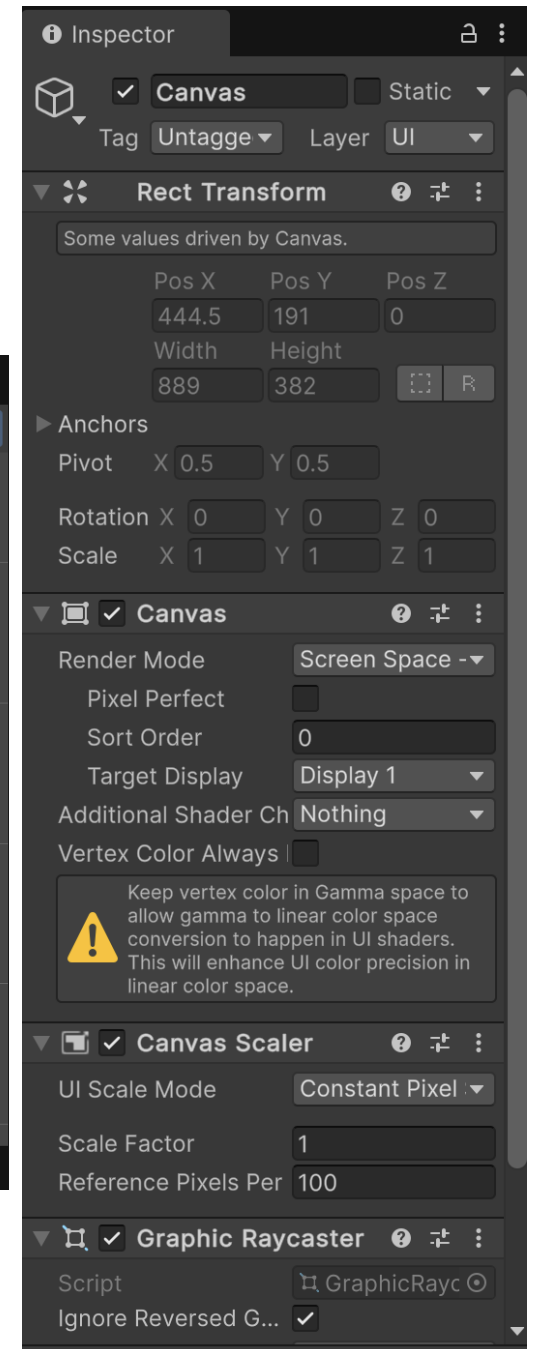
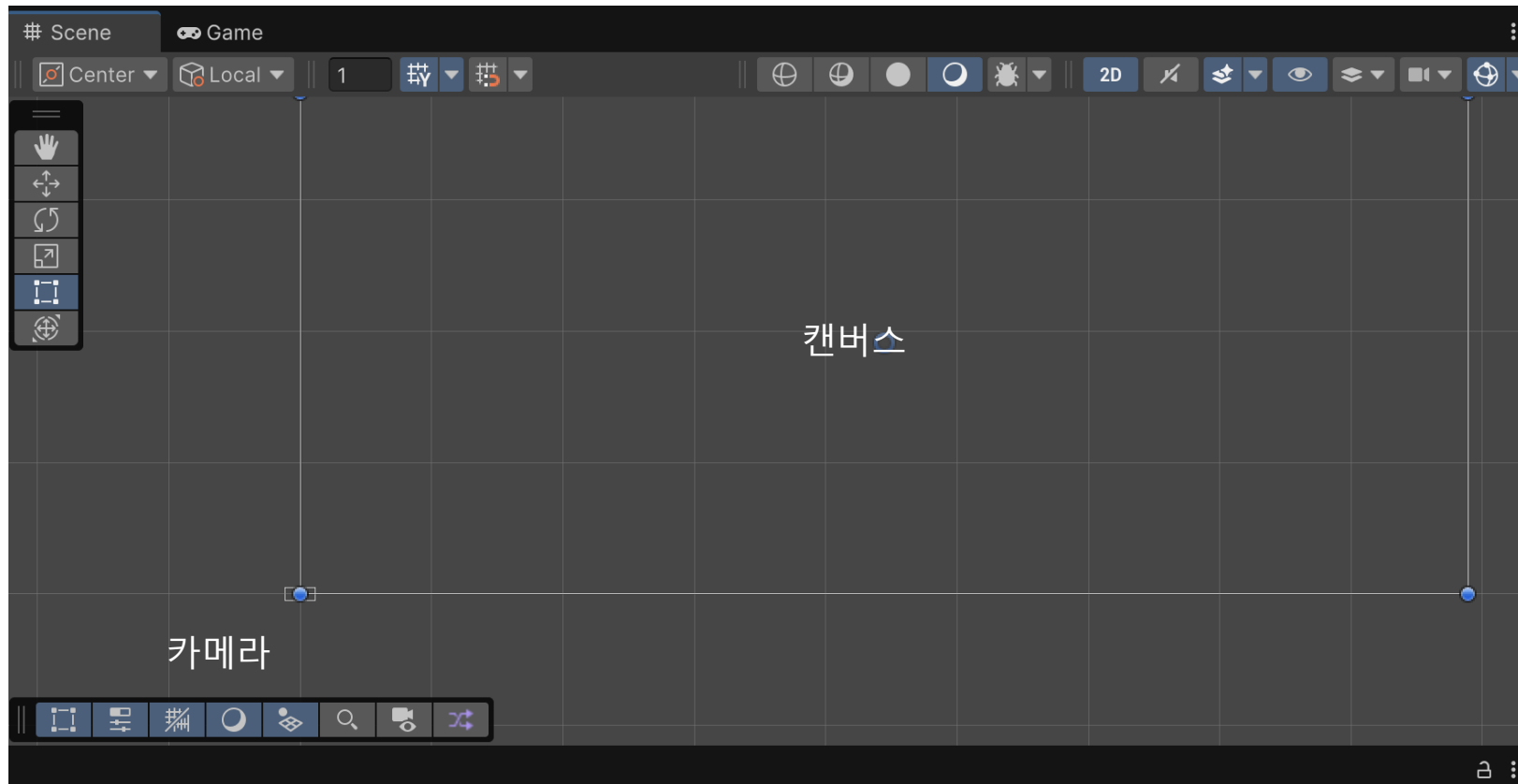
캔버스(Canvas)

- Hierarchy
 - Create → UI → Canvas
- UI 요소들이 캔버스에 실림
 - 캔버스는 스크린 공간에 존재
- 복수의 캔버스도 설치 가능



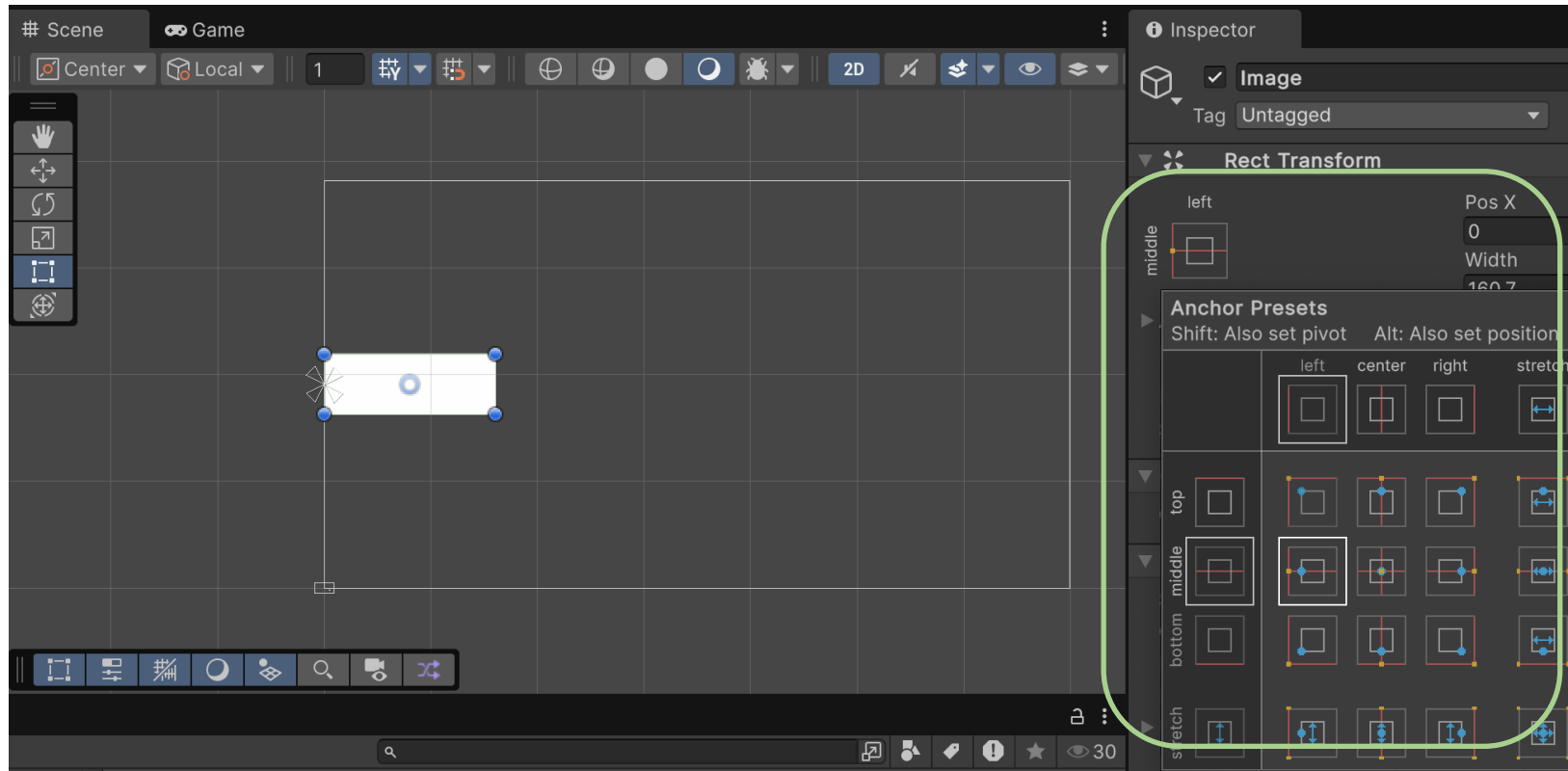
카메라 공간과 캔버스 공간

캔버스는
일반적 게임 객체의
변환과 다른 특성을 가짐

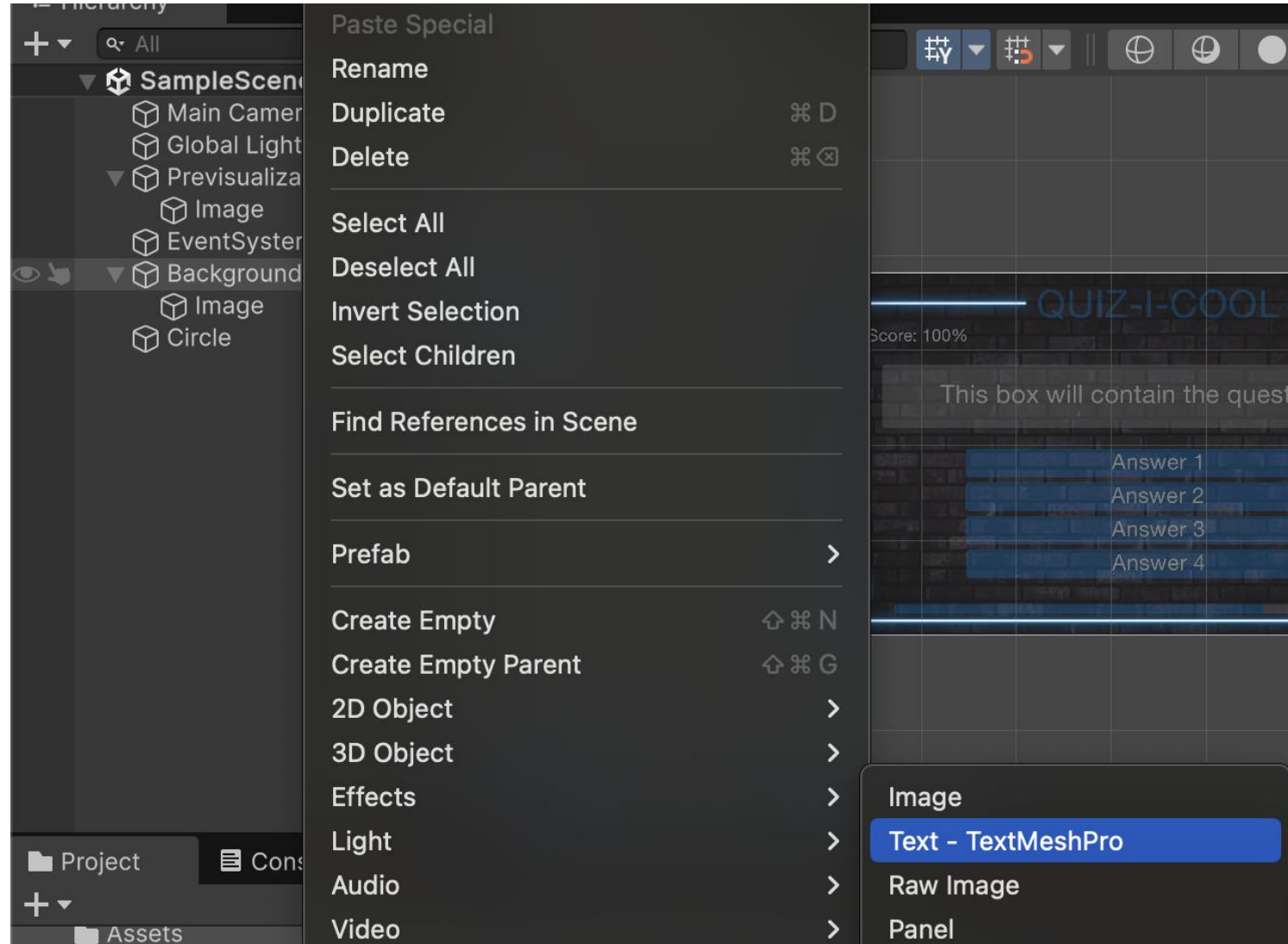


Anchor와 Pivot 테스트해 보기

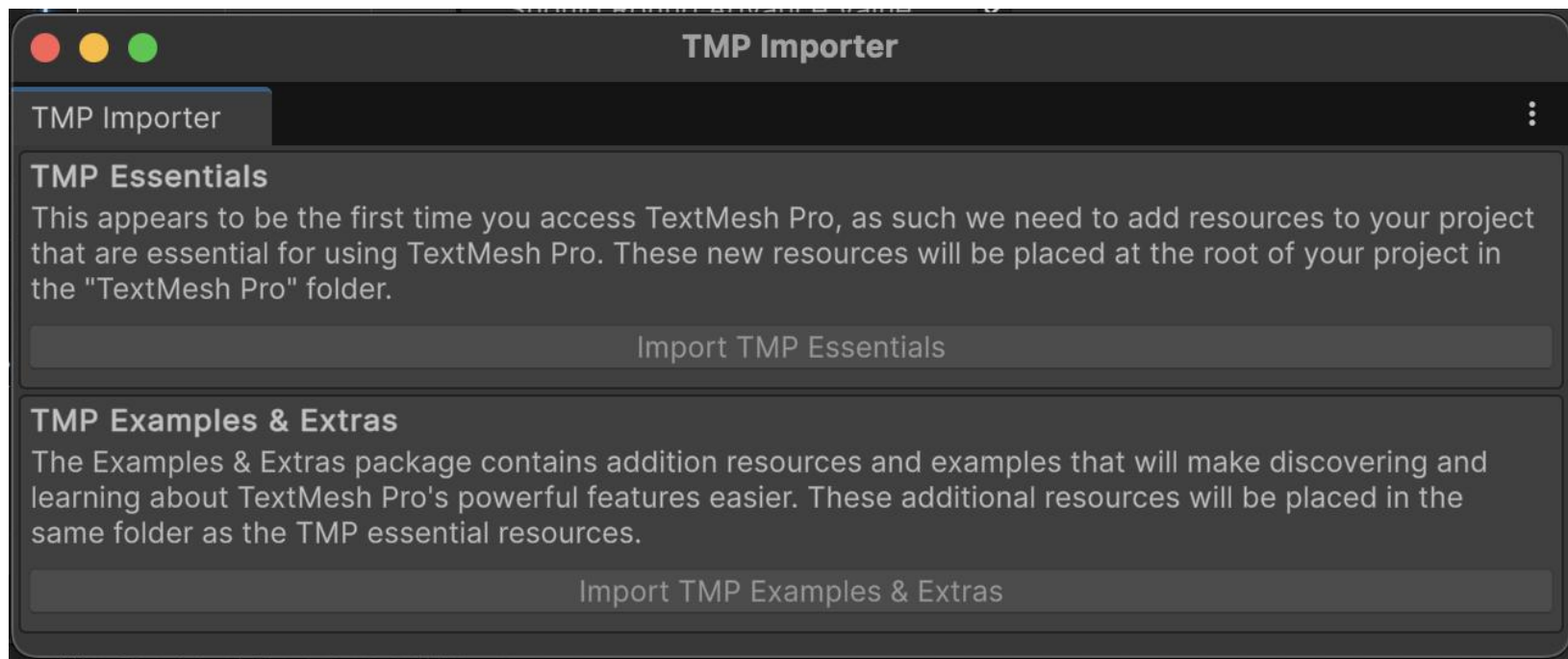
- Anchor Preset + (Shift / Alt)



배경 캔버스에 텍스트 메시 프로를 추가한다



Text Mesh Pro를 처음 사용할 경우 필요한 자원 가져오기



스크립터블 객체(scriptable objects)

- 일종의 데이터 컨테이너
- 스크립트 밖에서 데이터를 유지
- 데이터를 한 곳에 저장함으로써 메모리 절약 가능
- 게임 객체에 연결할 필요 없음
- 경량이며 사용이 편리
- 일관성을 유지하기 위한 템플릿으로 기능
- 사용가능 예
 - RPG 게임에서의 무기 스탯
 - 카드 게임의 카드 정보 등

* 여기서는 질문 데이터

사용 방법의 개념

코드: quiz.cs

스크립터블 객체

GetQuestionData()

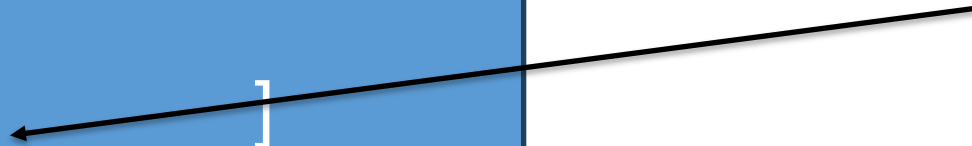
[]

DisplayQuestion()
CheckAnswer()

Question 1

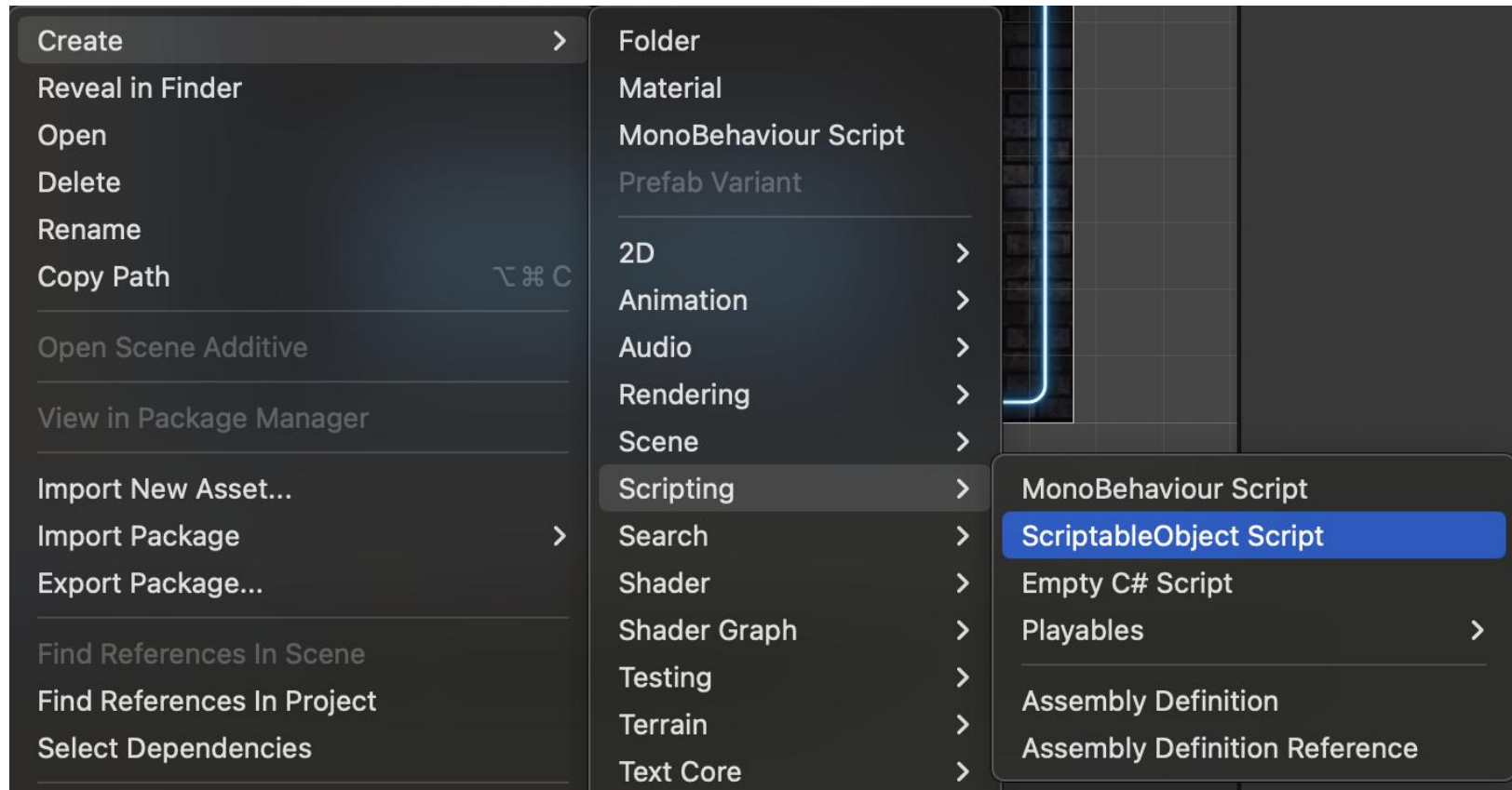
Question 2

Question 3



Scriptable Object 코드 작성

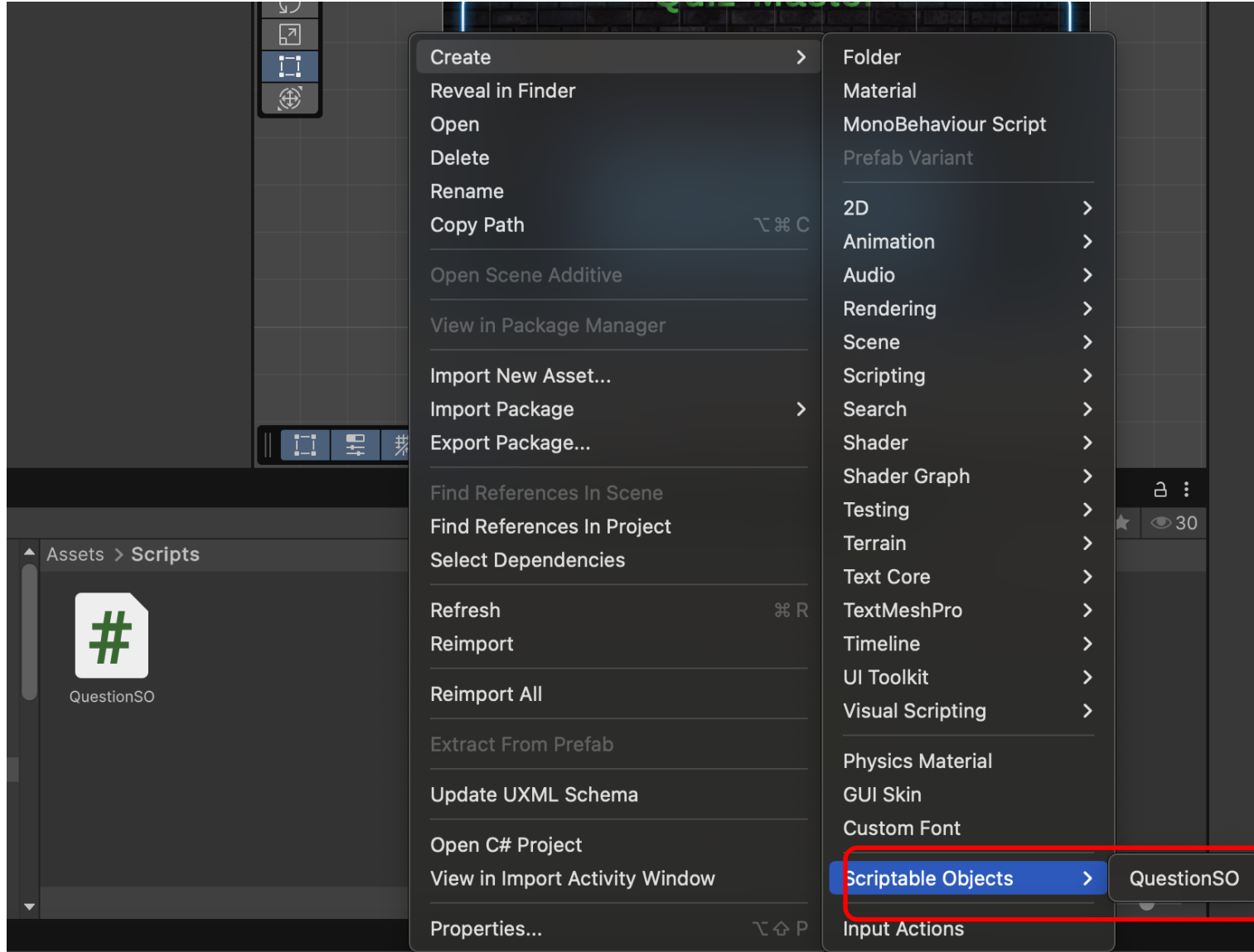
- Scripts 폴더 내에 스크립트 생성. (ScriptableObject Script로 생성)
 - QuestionSO.cs



기본 코드의 모양이 MonoBehaviour와 다름

```
1  using UnityEngine;
2
3  [CreateAssetMenu(fileName = "QuestionSO", menuName = "Scriptable Objects/QuestionSO")]
   0 references
4  public class QuestionSO : ScriptableObject
5  {
6
7  }
8  |
```

스크립터블 객체를 생성할 수 있게 됨



현재로서는 특별한 정보가 담기지
않음

타이머가 할 일

게임이 어떤 상태인지 확인

- 답변을 기다리는 상태
- 정답을 보여주는 상태

타이머가 시간 측정을 하도록 함

타이머 이미지의 상태를 타이머 상태에 맞춰 변경

타이머가 다 돌았으면 게임 상태 변경

Timer.cs

```
using UnityEngine;
using UnityEngine.UIElements;

0 references
public class Timer : MonoBehaviour
{
    2 references
    [SerializeField] float timeToCompleQuestion = 30f;
    2 references
    [SerializeField] float timeToShowCorrectAnswer = 10f;

    1 reference
    public bool loadNextQuestion;
    2 references
    public float fillFraction;
    3 references
    public bool isAnsweringQuestion;

    8 references
    float timerValue;

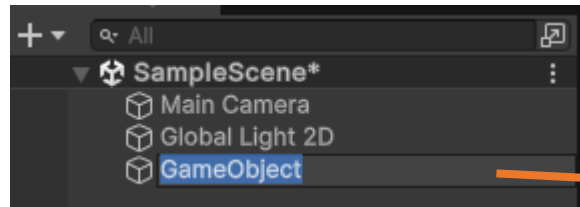
    0 references
    void Update()
    {
        UpdateTimer();
    }

    0 references
    public void CancelTimer()
    {
        timerValue = 0;
    }
}
```

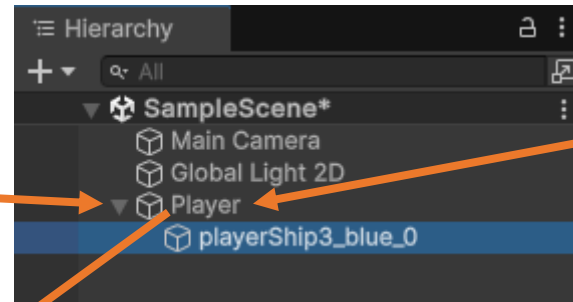
```
public void UpdateTimer()
{
    timerValue -= Time.deltaTime;

    if(isAnsweringQuestion)
    {
        if(timerValue > 0 )
        {
            fillFraction = timerValue / timeToCompleQuestion;
        }
        else
        {
            isAnsweringQuestion = false;
            timerValue = timeToShowCorrectAnswer;
        }
    }
    else // showing correct answer
    {
        if (timerValue > 0 )
        {
            fillFraction = timerValue / timeToShowCorrectAnswer;
        }
        else
        {
            isAnsweringQuestion = true;
            timerValue = timeToCompleQuestion;
            loadNextQuestion = true;
        }
    }
}
```


Prefab 제작 1 – Player



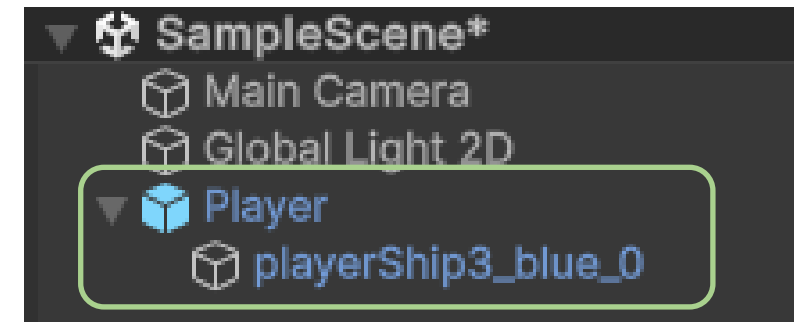
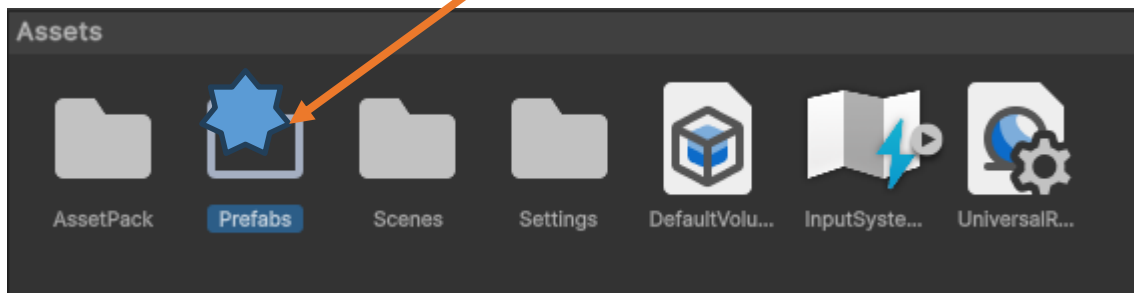
1. Rename



2. Drag & Drop

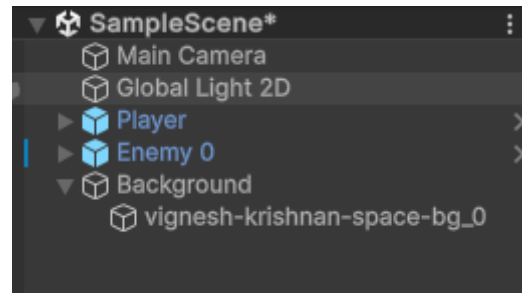
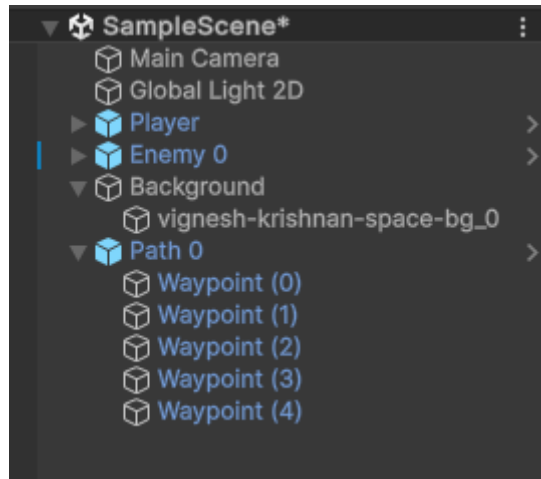


3. Create a folder for Prefabs and Drag & Drop "player"



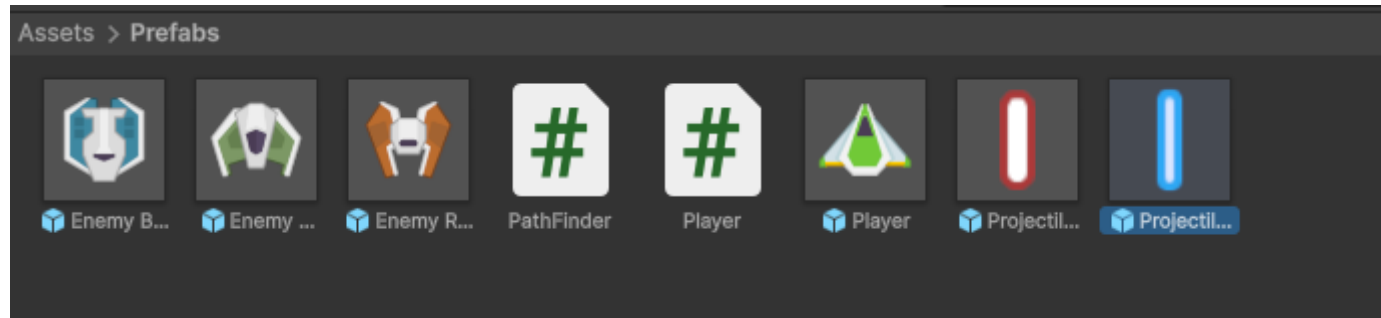
4. Prefab in Hierarchy

Prefab 된 객체는 없애도 됨



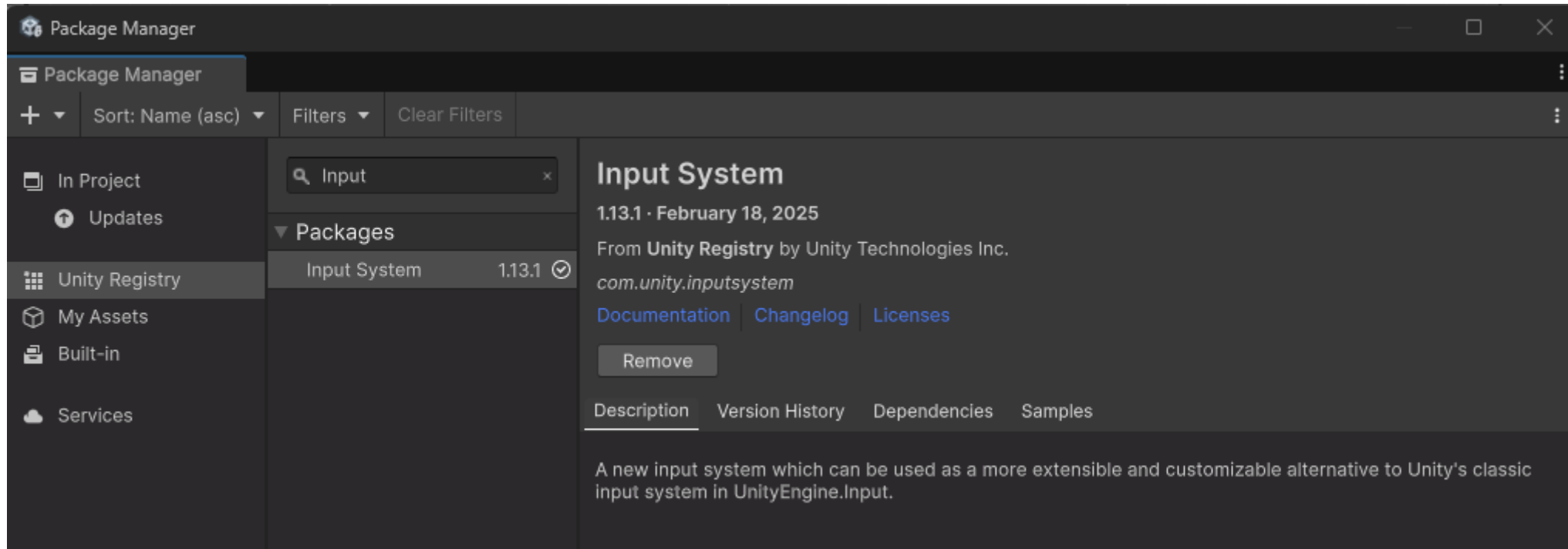
발사체를 프리팹으로

- 프리팹 폴더에 끌어다 놓고 Hierarchy에서 삭제



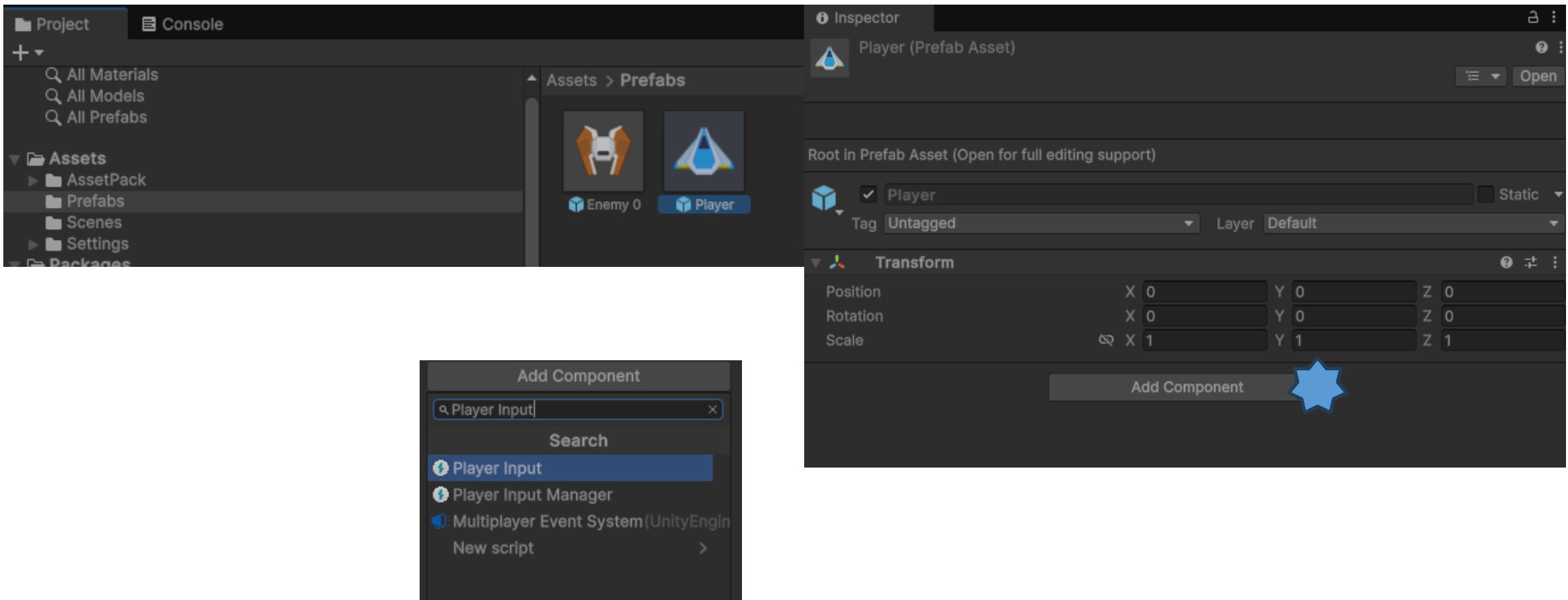
입력 시스템 - New Input System을 사용해 보자

- Windows → Package Manager
 - Input System 패키지 설치를 확인 (없으면 설치)



입력 시스템을 플레이어에 적용

- Hierarchy에 있는 특정 객체가 아니라 Prefab을 변경



OnMove에 의해 실행될 내용 담기

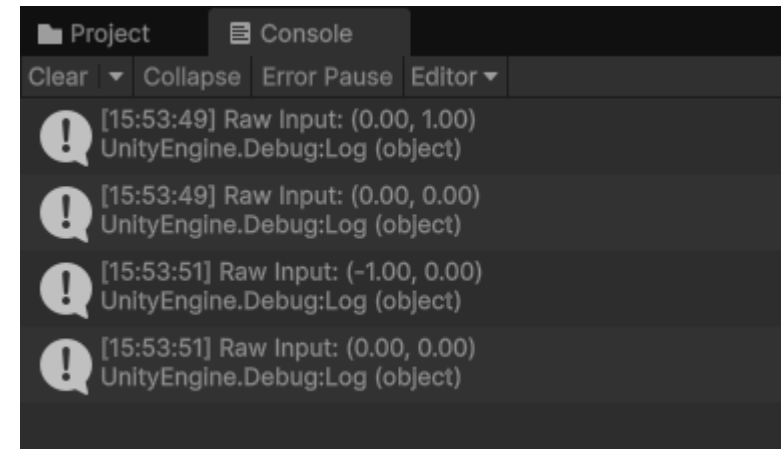
```
using UnityEngine;

using UnityEngine.InputSystem; // Required for Input System

public class Player : MonoBehaviour
{
    private Vector2 rawInput; // Store the raw input value

    void Update()
    {
    }

    void OnMove(InputValue value)
    {
        rawInput = value.Get<Vector2>();
        Debug.Log("Raw Input: " + rawInput);
    }
}
```



Dynamic vs. Kinematic

- Dynamics (동역학)

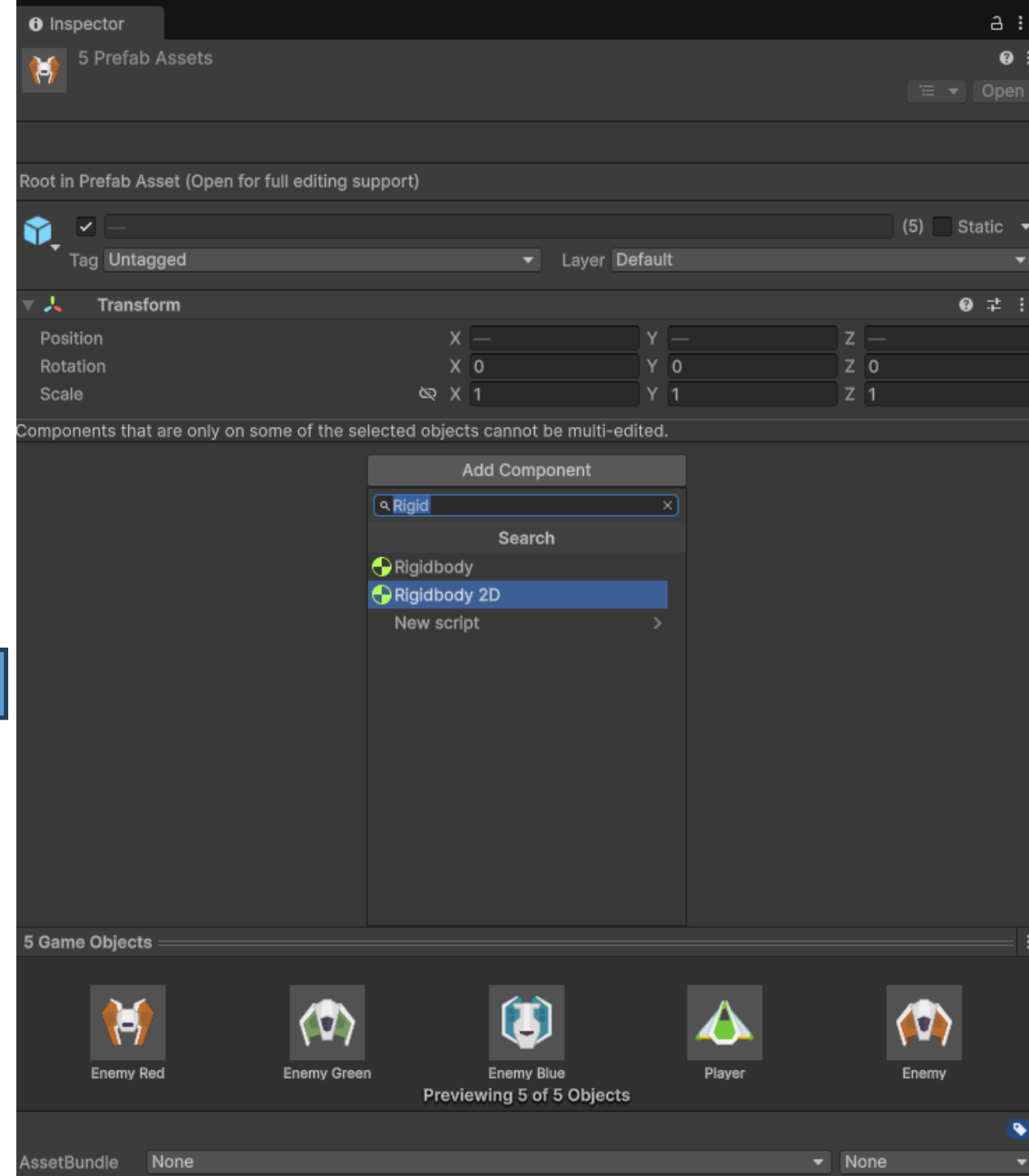
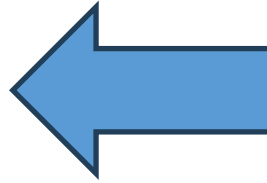
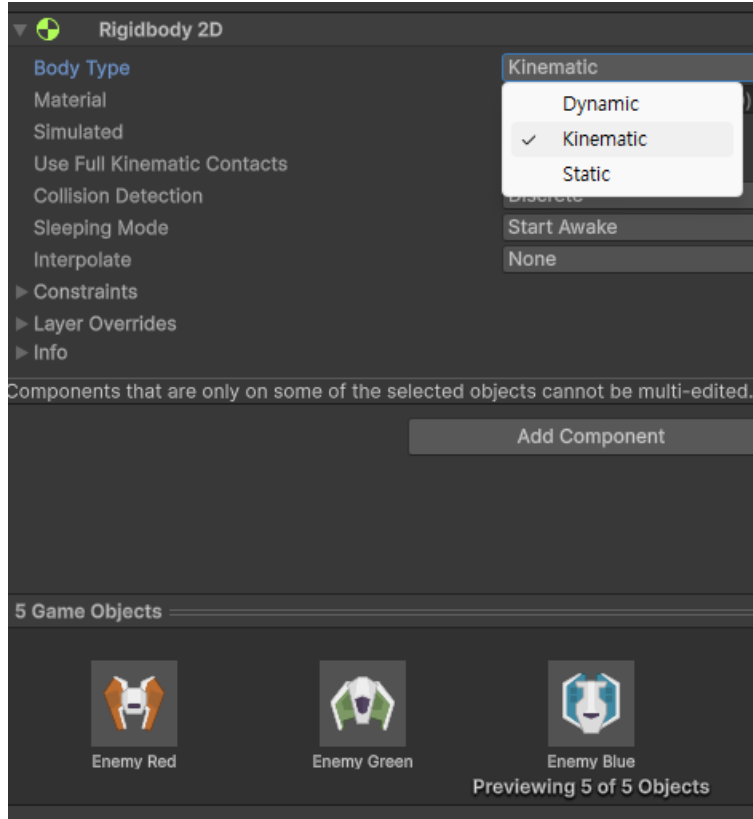
- 물리 시뮬레이션에 의해 힘이 계산되고 그 힘에 의해 물체가 가속/이동
 - 물체의 위치를 명시적으로 설정하지 않고 위치를 결정하는 조건을 지정
 - 물리 법칙에 따라 움직임 (자연스러운 물리 시뮬레이션)
 - 제어의 어려움 존재

- Kinematics (운동학)

- 물체의 움직임을 표현하는 곡선을 다루는 분야
 - 물체의 위치를 명시적으로 설정
 - 제어가 용이함

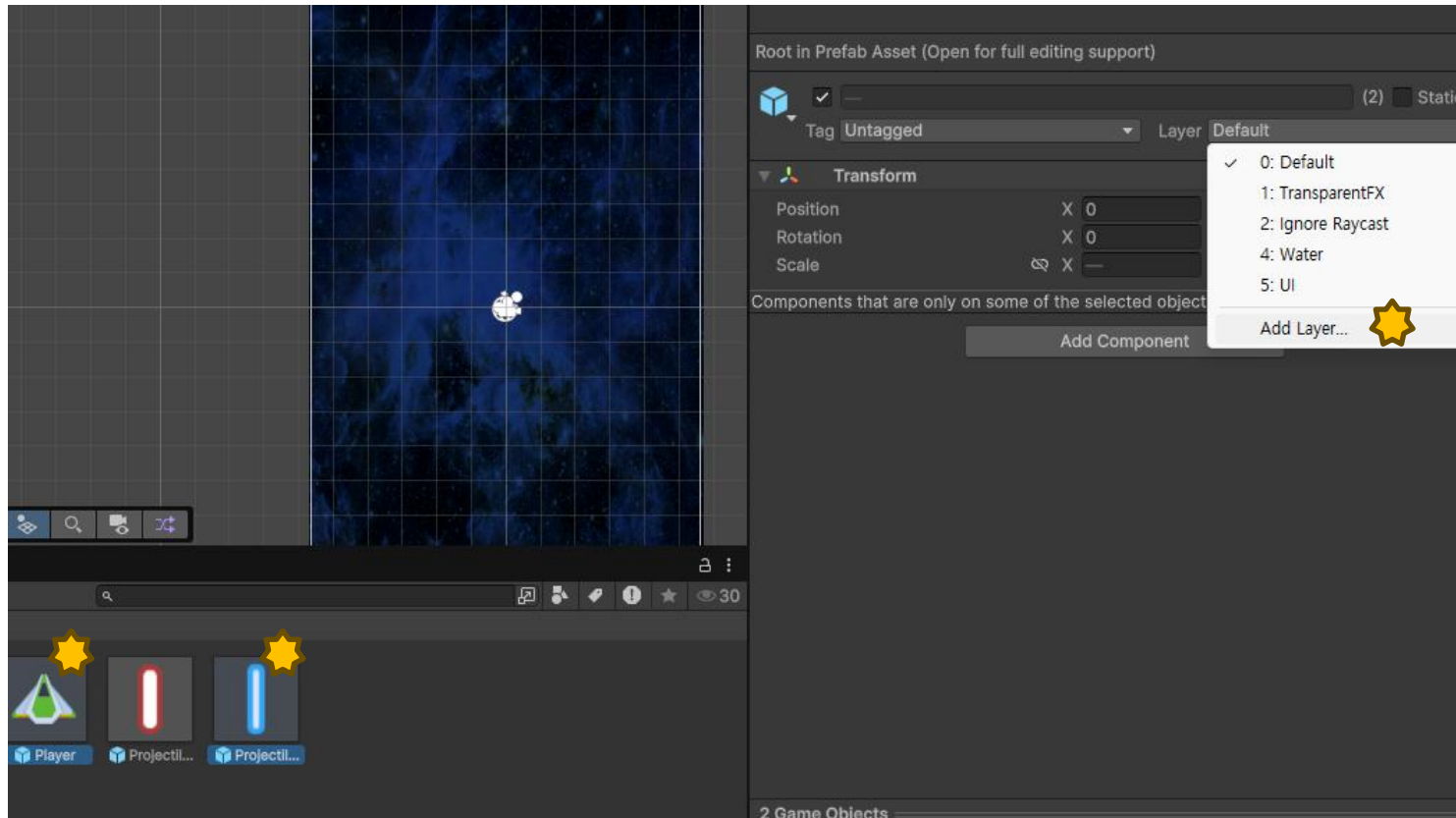
Rigidbody 2D 추가

- Body Type 변경
 - Dynamic → Kinematic



Layer

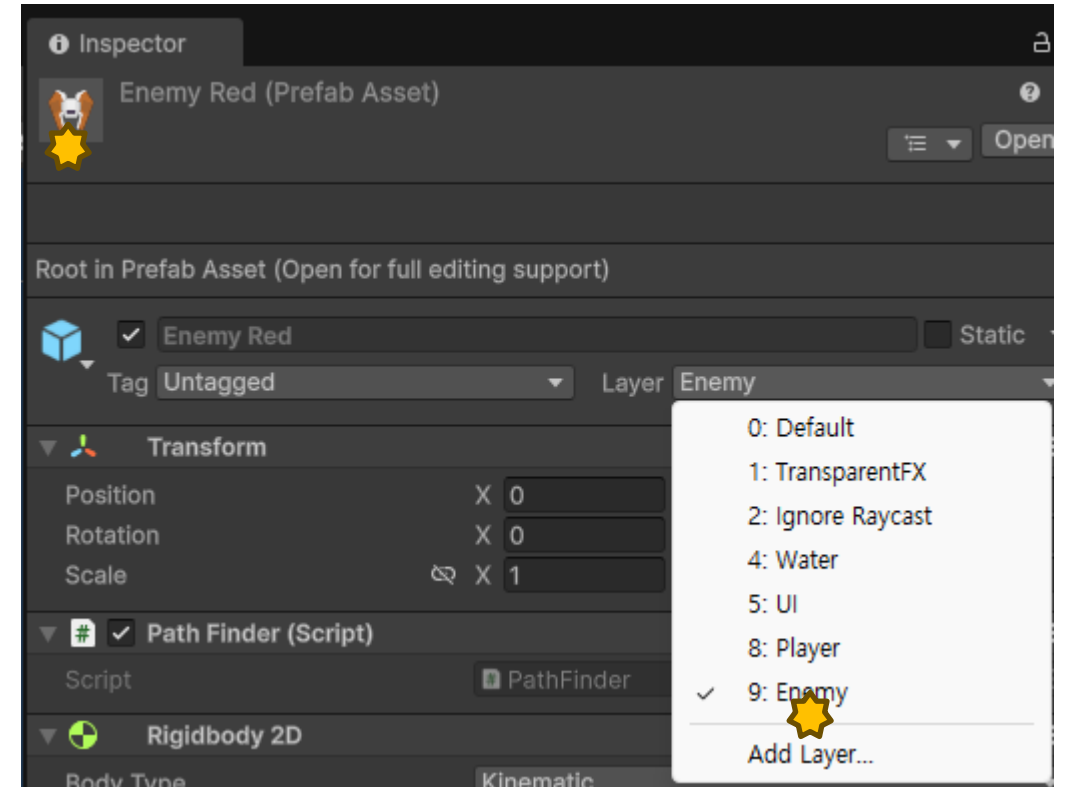
- 플레이어와 플레이어 발사체의 레이어 설정



▼ Layers	
Builtin Layer 0	Default
Builtin Layer 1	TransparentFX
Builtin Layer 2	Ignore Raycast
User Layer 3	
Builtin Layer 4	Water
Builtin Layer 5	UI
User Layer 6	
User Layer 7	
User Layer 8	Player 

Layer

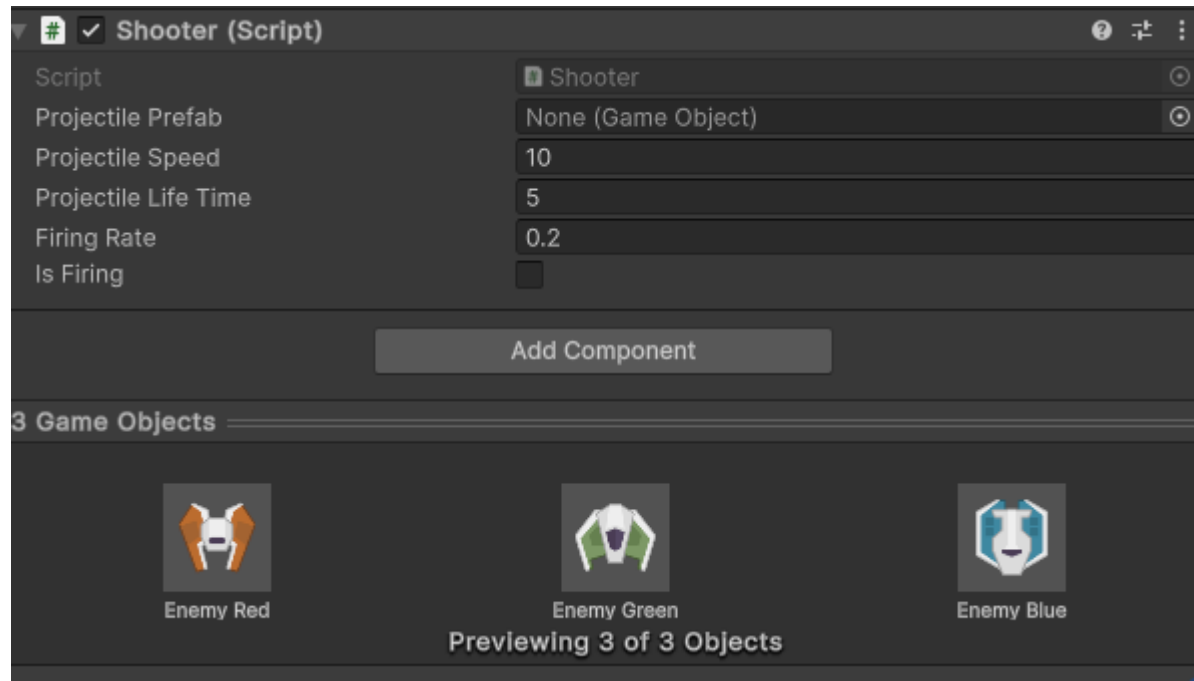
- 적과 적 발사체도 레이어 설정
 - Layer 추가 이후에는 지정을 해야 함



적의 슈팅

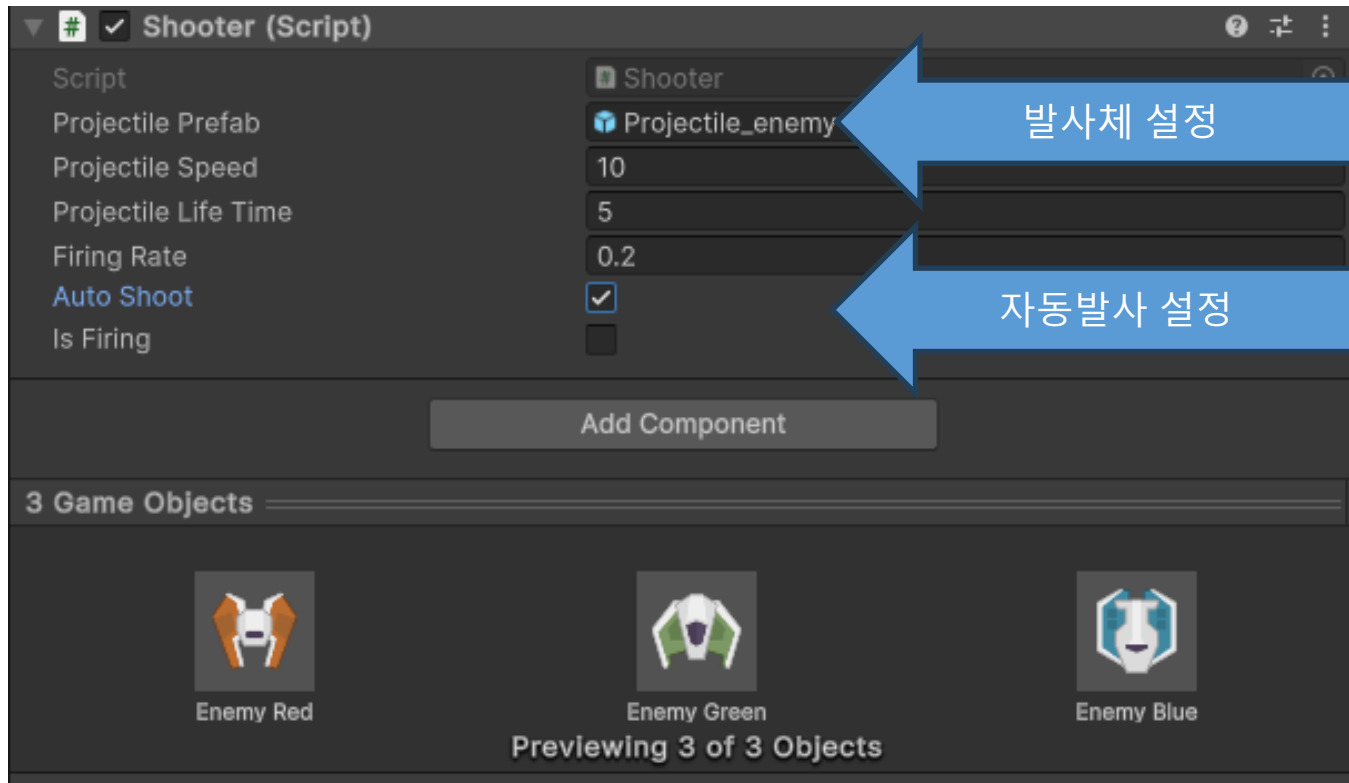
- 이미 만들어 놓은 스크립트 활용

Shooter.cs를 적에 적용



적은 입력이 아니라 자동으로 발사

- Shooter.cs 수정: 자동 발사 옵션 추가



```
public class Shooter : MonoBehaviour
{
    1 reference
    [SerializeField] GameObject projectilePrefab;
    1 reference
    [SerializeField] float projectileSpeed = 10f;
    1 reference
    [SerializeField] float projectileLifeTime = 5f;
    1 reference
    [SerializeField] float firingRate = 0.2f;
    1 reference
    [SerializeField] bool autoShoot = false;

    5 references
    public bool isFiring = false;
    5 references
    Coroutine firingRoutine;

    0 references
    void Start()
    {
        if (autoShoot)
        {
            isFiring = true;
        }
    }
}
```

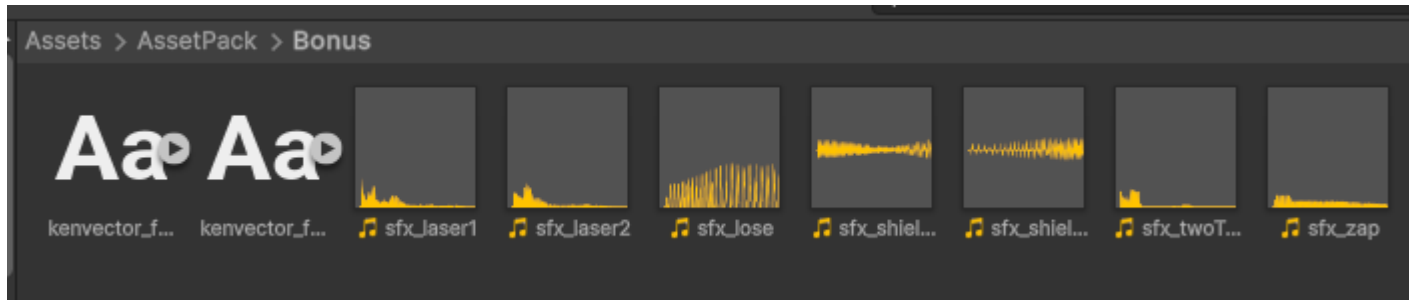
적기의 발사 속도 조정 – Coroutine!

```
IEnumerator FireContinuously()
{
    while(true)
    {
        // firing action
        GameObject missile = Instantiate(projectilePrefab,
            transform.position, Quaternion.identity);

        Rigidbody2D rb = missile.GetComponent<Rigidbody2D>();
        if (rb != null) ...

        Destroy(missile, projectileLifeTime);
        float delay = autoShoot ? 4f : 1f;
        yield return new WaitForSeconds(delay*firingRate);
    }
}
```

음향 효과 – 애셋 준비



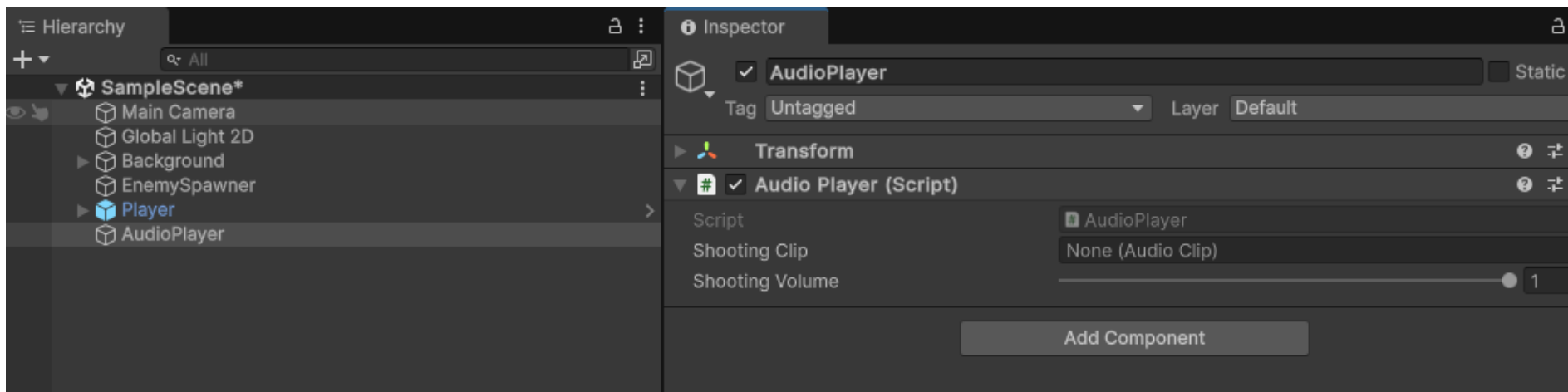
AudioPlayer.cs

- Hierarchy에 AudioPlayer 생성
 - AudioPlayer.cs 연결

```
using UnityEngine;

public class AudioPlayer : MonoBehaviour
{
    [SerializeField] AudioClip shootingClip;
    [SerializeField] [Range(0f, 1f)] float shootingVolume = 1f;

    public void PlayShootingClip()
    {
        if (shootingClip != null)
        {
            AudioSource.PlayClipAtPoint(
                shootingClip,
                Camera.main.transform.position,
                shootingVolume
            );
        }
    }
}
```



싱글톤 패턴으로 구현하는 게임 매니저

- 싱글톤 - 유일한 객체를 만들기 위한 디자인 패턴
 - 게임관리자
 - 두 개의 관리자가 있을 이유가 없음
- 게임 매니저
 - 씬의 전환
 - 스코어의 관리
 - 옵션의 관리
- 통합적으로 게임을 관리하는 하나의 매니저만 필요
 - 단 하나만 존재하는 **유일성**과 모두가 사용할 수 있는 **접근성**이 특징
 - 싱글톤 패턴으로 구현하는 것이 바람직

싱글톤 유일성 구현 방법

- 유일성을 유지하기 위한 변수 설정

```
public class GameManager : MonoBehaviour {  
    private static GameManager instance;
```

```
void Awake()  
{  
    if(instance != this)  
    {  
        Destroy(gameObject);    유일성 유지  
    }  
}
```

싱글톤 접근성 구현 방법

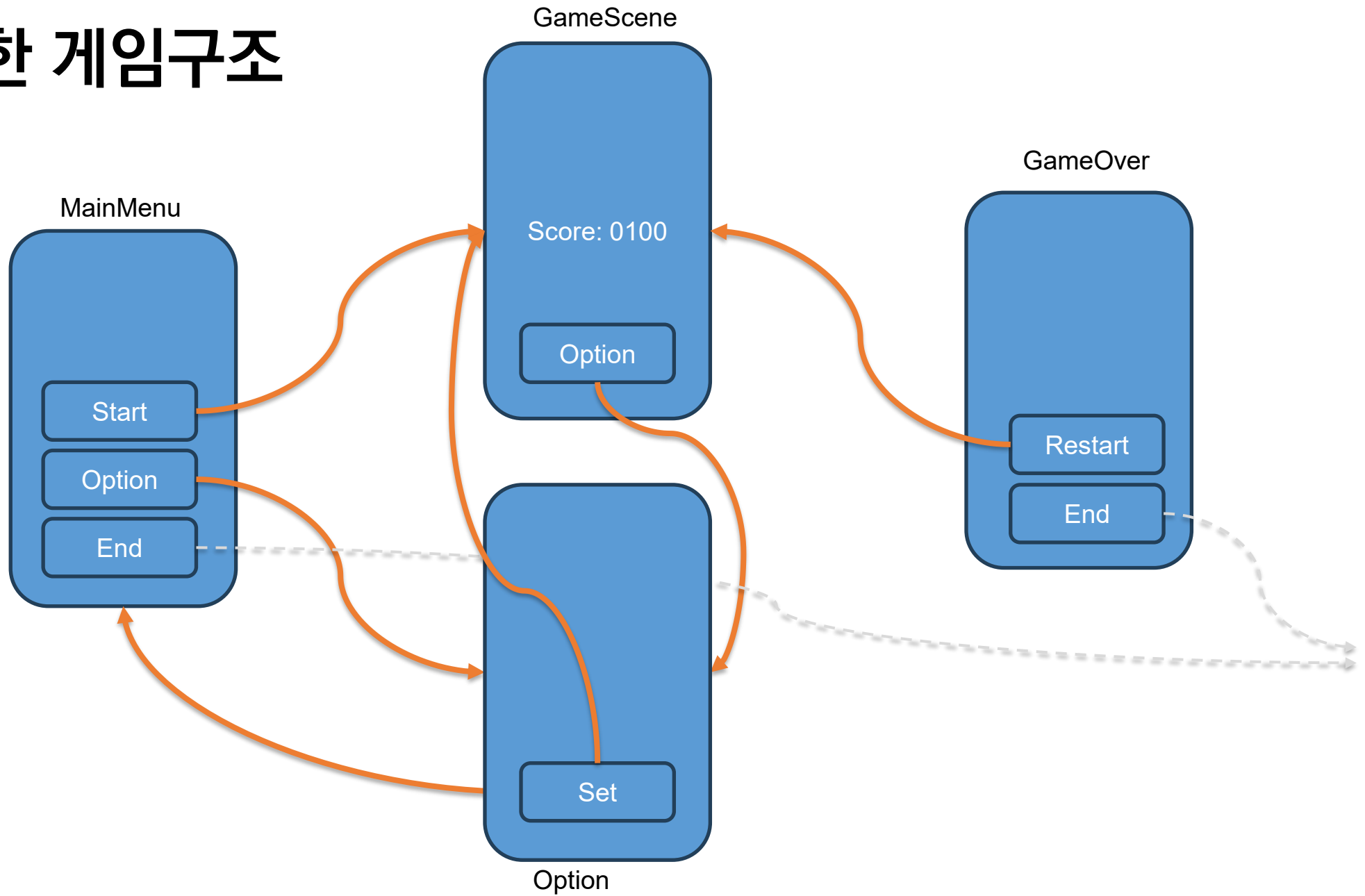
- 외부 호출 변수를 public으로 구현

```
public void DoSomething(... args ...)  
{  
    .... Do something  
}
```

```
public class PlayerCtrl : MonoBehaviour  
{  
    public void NeedSomething ()  
    {  
        GameManager.Instance.DoSomething(...);  
    }  
}
```

어디서든 접근

간단한 게임구조



GameManager.cs

```
public class GameManager : MonoBehaviour
{
    private static GameManager instance;
    public static GameManager Instance
    {
        get
        {
            if (instance == null)
            {
                return null;
            }
            return instance;
        }
    }
}
```

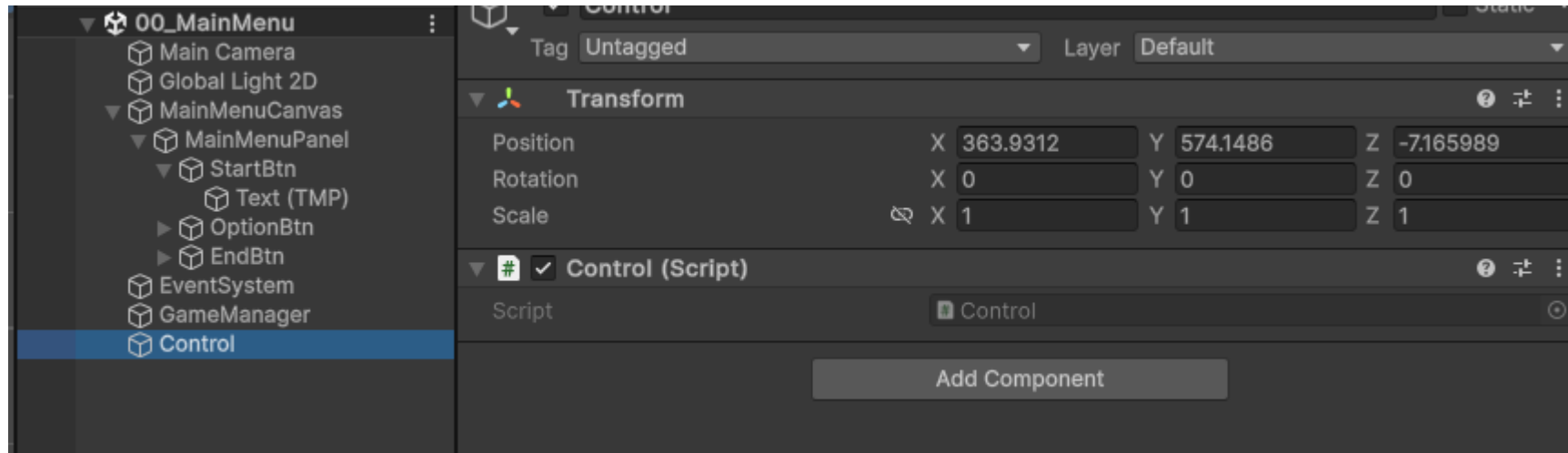
```
private void Awake()
{
    if (instance == null)
    {
        instance = this;
        DontDestroyOnLoad(gameObject);
    }
    else
    {
        Destroy(gameObject);
    }
}
```

Game Manager에 장면 전환 기능 추가

```
using UnityEngine.SceneManagement;
```

```
public void LoadScene(string sceneName)  
{  
    SceneManager.LoadScene(sceneName);  
}
```

MainMenu에 Control 추가



Control.cs 연결

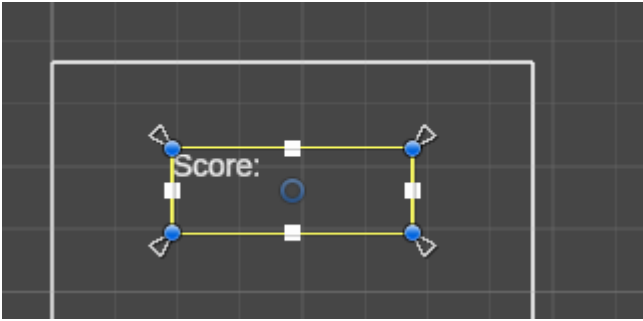
```
public class Control : MonoBehaviour
{
    // Start is called once before the first execution of Update after the
    // 0 references
    public void ChangeScene(String sceneName)
    {
        if (GameManager.Instance != null)
        {
            GameManager.Instance.LoadScene(sceneName);
        }
        else
        {
            Debug.LogError("GameManager instance is not available.");
        }
    }
    // 0 references
    void Start()
```

Score 관리는? 게임 매니저에서

```
public class GameManager : MonoBehaviour
{
    4 references
    private static GameManager instance;
    4 references
    public String caller;
    1 reference
    public int score;
}
```

```
private void Awake()
{
    if (instance == null)
    {
        instance = this;
        score = 0; // Initialize score
        DontDestroyOnLoad(gameObject);
    }
    else
    {
        Destroy(gameObject);
    }
}
```


Score Text에 ScoreDisplay.cs 연결

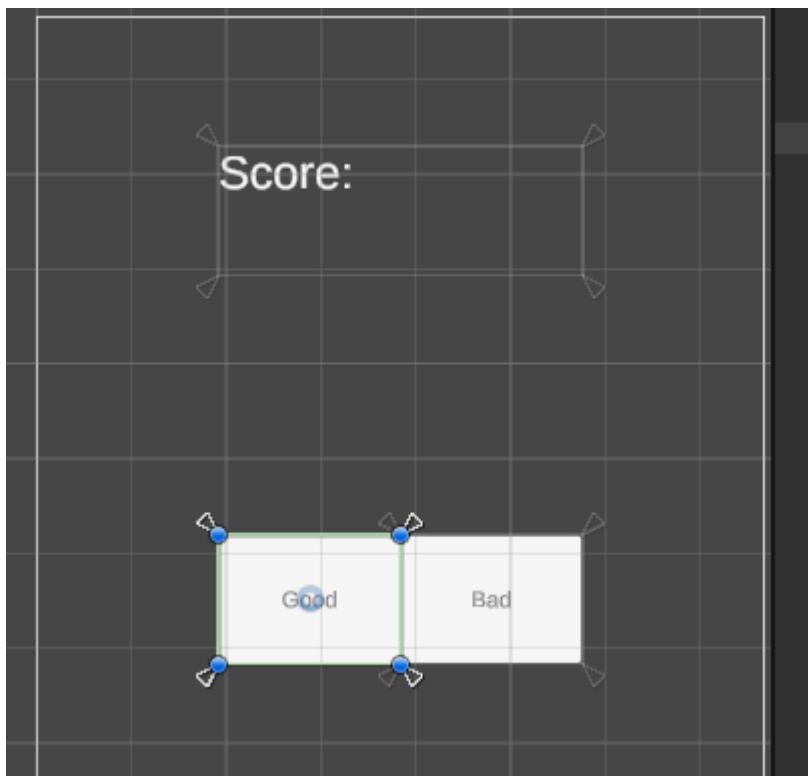


```
using TMPro;
using UnityEngine;

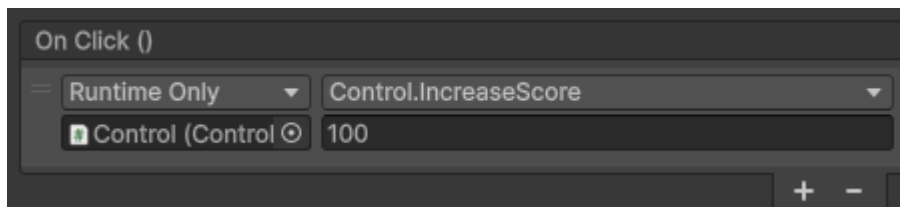
0 references
public class ScoreDisplay : MonoBehaviour
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    0 references
    void Start()
    {
    }

    // Update is called once per frame
    0 references
    void Update()
    {
        if (GameManager.Instance == null)
        {
            Debug.LogError("GameManager instance is not available.");
            return;
        }
        gameObject.GetComponent<TextMeshProUGUI>().text = "Score: " + GameManager.Instance.score.ToString();
    }
}
```

점수 증가



```
0 references
public void IncreaseScore(int amount)
{
    if (GameManager.Instance != null)
    {
        GameManager.Instance.score += amount;
    }
    else
    {
        Debug.LogError("GameManager instance is not available.");
    }
}
```



Score 관리가 이루어짐

